capital

Web Services Development

Software Version 2015.1

February 2016

© 2016 **Mentor Graphics Corporation**
All rights reserved.

This document contains information that is proprietary to Mentor Graphics Corporation. The original recipient of this document may duplicate this document in whole or in part for internal business purposes only, provided that this entire notice appears in all copies. In duplicating any part of this document, the recipient agrees to make every reasonable effort to prevent the unauthorized use and distribution of the proprietary information.

This document is for information and instruction purposes. Mentor Graphics reserves the right to make changes in specifications and other information contained in this publication without prior notice, and the reader should, in all cases, consult Mentor Graphics to determine whether any changes have been made.

The terms and conditions governing the sale and licensing of Mentor Graphics products are set forth in written agreements between Mentor Graphics and its customers. No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Mentor Graphics whatsoever.

MENTOR GRAPHICS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS MATERIAL INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

MENTOR GRAPHICS SHALL NOT BE LIABLE FOR ANY INCIDENTAL, INDIRECT, SPECIAL, OR CONSEQUENTIAL DAMAGES WHATSOEVER (INCLUDING BUT NOT LIMITED TO LOST PROFITS) ARISING OUT OF OR RELATED TO THIS PUBLICATION OR THE INFORMATION CONTAINED IN IT, EVEN IF MENTOR GRAPHICS CORPORATION HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

RESTRICTED RIGHTS LEGEND 03/97

U.S. Government Restricted Rights. The SOFTWARE and documentation have been developed entirely at private expense and are commercial computer software provided with restricted rights. Use, duplication or disclosure by the U.S. Government or a U.S. Government subcontractor is subject to the restrictions set forth in the license agreement provided with the software pursuant to DFARS 227.7202-3(a) or as set forth in subparagraph (c)(1) and (2) of the Commercial Computer Software - Restricted Rights clause at FAR 52.227-19, as applicable.

Contractor/manufacturer is:
Mentor Graphics Corporation
8005 S.W. Boeckman Road, Wilsonville, Oregon 97070-7777.
Telephone: 503.685.7000
Toll-Free Telephone: 800.592.2210
Website: *www.mentor.com*
SupportNet: *www.mentor.com/supportnet*
Contact Your Technical Writer: *www.mentor.com/supportnet/documentation/reply_form.cfm*

TRADEMARKS: The trademarks, logos and service marks ("Marks") used herein are the property of Mentor Graphics Corporation or other third parties. No one is permitted to use these Marks without the prior written consent of Mentor Graphics or the respective third-party owner. The use herein of a third-party Mark is not an attempt to indicate Mentor Graphics as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A current list of Mentor Graphics' trademarks may be viewed at: *www.mentor.com/terms_conditions/trademarks.cfm*.

End-User License Agreement: You can print a copy of the End-User License Agreement from: *www.mentor.com/terms_conditions/enduser.cfm*.

Table Of Contents

Introduction 3
Integration patterns 3
Web services protocol 4
Capital Web Services exposed in CIS 5
How to configure CIS 5
Release Designs as Task ..... **Error! Bookmark not defined.**
Using the Build List Importer service ..... **Error! Bookmark not defined.**
Using the Build List exporter service ..... **Error! Bookmark not defined.**
How to invoke a CIS web service 14
How to build an example Java client 14
How to run an example Java client 15
Using the Projects list service 15
Using the Designs list service 16
Using the Diagrams list service 17
Using the SVG diagram generator service 18
Using the DXF diagram generator service 19
Using the SVG symbol generator service 20
Using the Design exporter service 22
Using the Option information exporter service 23
Using the Object Type information exporter service 24
Using the Usage Definitions exporter service 27
Using the Usage Definitions Import service 28
Using the Security data exporter service 29
Using the Design release status update service 30
Using the Wiring properties update web service 31
Using the Option information importer service 34
Using the Object Type information importer service 35
Using the Library importer service 36
Using the Design revisionning service 36
Using the Project Template exporter service 37
Using the System Data exporter service 38
Using the Library Parts exporter service 39
Using the Design Bridge Out service 40
Using the Design importer service 40
Using the Symbol exporter service 41
Using the Symbol importer service 42
Using the Language Dictionary Export service 42
Using the Language Dictionary importer service 43
Using the EngineeringChangeOrders exporter service 43
Using the EngineeringChangeOrders importer service 45
Using the Submit Task service 45
Using the List Tasks service 47
Using the Describe Task service 48
Using the Delete Task service 48
Using the Release Multiple Designs service 49
Using the Build List Release Level Change service 51
DRC as Task 52
Release Designs as Task 53
Using the BuildLists list service 54
Using the Build List Importer service 55
Using the Build List exporter service 56
Using the Batch Bridge In Task service 57

External Web Services deployed in CIS 60
How to configure CIS to expose external web services 60
How to deploy Axis web services..... 60
How to deploy other web services 62
How to build the example web service container application..... 63
How to run the example external web service 63
Architecture of the example external web service 63
Capital web service hooks 65
How to configure a hook..... 65
How to implement a hook web service 66
How to implement the design release status hook 67
How to implement the design attributes update hook 68
How to implement the library component update hook..... 71
How to implement the library component status change hook 73
How to implement the engineering change order release status hook..... 74
How to implement the Build List release status hook 76

Introduction

This document describes the process for integrating external applications with Capital, using web services.
The document can be found in $CAPITAL_HOME/doc/webservice (where $CAPITAL_HOME is the directory that contains your Capital installation) and is illustrated by a number of fully functional examples. Each example comes with

Integration patterns

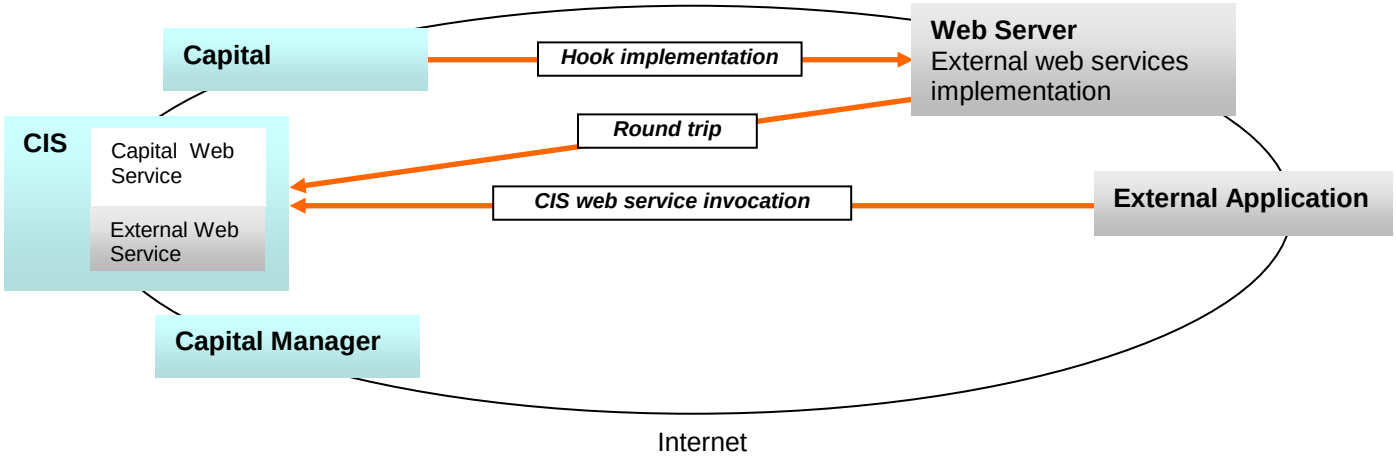
Capital currently includes two web service based mechanisms for an external application to integrate:

- by invoking web services exposed in the Capital Integration Server (CIS)
- by implementing external web services invoked by Capital through configured hooks

Note that the second mechanism enables round-trip flows where the external web service implementation in turn invokes a CIS web service.

It is also possible to deploy your own external web services in CIS, but those services will have no means of accessing Capital logic, other than invoking CIS native web services.

The figure below illustrates the possible integration patterns.



Web services protocol

Capital web services are based on an XML document exchange protocol, using SOAP messages over HTTP:

- SOAP is not used to transport fine grain remote method invocations
- An XML document is sent as the XML payload of a SOAP request message to a service
- The service processes the request XML payload and sends back an optional response which also includes an XML document payload. The response can also contain a SOAP fault to account for service failures.
- Some services will make use of SOAP attachments, in which case the data payload is not contained within the SOAP body, but within attachments of various formats (e.g. gzip, XML, etc…). In this case, the SOAP body can contain other information such as options to handle the payload, or can be empty.

The core of this protocol, i.e. the interface between client and service, are the XML schemas on which the request and response payloads are based.
These schemas are published in $CAPITAL_HOME/dtd, unless they are provided as standards on the internet (e.g. SVG).

Capital web services support two basic types of transactions:

- Synchronous (request-response) where the client blocks until a response (which can be a fault) is received from the service
- Asynchronous (fire-and-forget) where the client resumes immediately after the request is sent, and does not process any response

However the web service itself has no knowledge of this transaction choice, which has to be implemented on the client: the service always works in a synchronous mode, i.e. the client blocks until the service has completed and sends the response (which can be empty). The choice of transaction is purely client dependent:

- Synchronous / request-response: the client invokes the web service in the current thread, which blocks until the web service has completed and response has been received. The client then processes the response in the same thread.
- Asynchronous / fire-and-forget: the client forks a secondary thread in which the web service is invoked. The main thread resumes immediately and the secondary thread blocks until the web service has completed, but is ignored.
- Asynchronous / request-response: Uses a secondary thread as in previous case, but the secondary thread wakes up when the web service has completed and response is received, to process the response.

Capital Web Services exposed in CIS

Capital provides web services:

- Exposed in the Capital Integration Server (CIS), for any third party to make use of
- Embedded in Capital applications, but not currently exposed to third parties – for instance to support dynamic integration with MCAD systems)

This chapter explains how to make use of Capital web services provided by CIS.

How to configure CIS

CIS is, like any other Capital application, a client of Capital Manager. As a web server, it listens on a port that has to be specified in the Capital configuration.

This port number will be prompted to the user as part of the Capital installation (specific CIS part of the installation), then will be persisted to the $CAPITAL_HOME/config/clientprops.xml file. The port number can subsequently be modified at any convenient time by manually editing the file as follows:

```
<webserver host="cis" port="49901" cisjettyroot="../adaptors/webapps"
embedded="false"/>
```

The *host* attribute specifies the host application of a specific web server and can take the following values:

- *cis* the host application is the Capital Integration Server
- *processing* the host application is the Capital Harness Processing Server
- *reporter* the host application is the Capital Enterprise Reporting Server

The *cisjettyroot* attribute specifies the directory for deployment of external web services within this web server (see DRC as Task

It is possible to run DRCs on any design, buildlist or SBOM by submitting task. You have to give appropriate application name to run and list of DRCs as part the task parameters.

The name of task is DRCTask and it takes a config file which has details of target and list of DRCs that should be run on the target. Please see the example file DRCProcessingTask.xml in {capital.home}\doc\webservice\data, which has list of all available DRCs.

The following is a sample config file for a DRC task:

```
<?xml version="1.0" encoding="UTF-8"?>
<drcsettings project="exampleproject" drcobject="UID6a6ebd-13e5c321c48-
87fecf32183b52214ad60e22ba7de368" result="$CHS_HOME/temp/drc_task_result.csv">
  <drctorun drcname="Logic.Checks.DanglingNetCheck" severity="Error" title="Net not terminating
at a pin"/>
  <drctorun drcname="Logic.Checks.DanglingShieldCheck" severity="Error" title="Shield termination
not terminating at a pin"/>
  <drctorun drcname="Logic.Checks.plugin:Java//com.mentor.drc.ExampleCustomDRC"
severity="Warning" title="Custom DRC"/>
/drcsettings>
```

Below is the explanation of each attribute in config xml file

Project: The name of the project on which the DRC task needs to run

DRCObject: This is uid of the design, buildlist or SBOM on which DRCs needs to run. UID can be obtained by looking at project xml file. You can also find design uid using listdesigns webservice

Result: This is the location where result of DRC run can be written.

DRCName: This is the unique identifier for a DRC. You can look at DRCProcessingTask.xml to find identifier for all available DRCs in capital

Severity: Severity of each DRC. The valid values for this are Error, Warning, Information

The following is a sample result csv file:

```
"Error","Invalid name: P801 is not on the list of valid Connector names","Naming","Invalid name
for component","7-Power Mirrors_Wiring","P801"

"Warning","Device DOOR ECU has no symbol","Inclusion","Missing symbol for device","7-Power
Mirrors_Wiring","DOOR ECU"
```

Release Designs as Task

It is possible to release any design by submitting the *ReleaseDesignsTask*. It takes an appropriate application name and a configuration file which has details of design revisions to be transitioned to target release level. Please see the example file ReleaseDesignsTask.xml in {capital.home}\doc\webservice\data, which has a sample format to release multiple designs.

```
<?xml version="1.0" encoding="UTF-8"?>
<project projectname="example1" annotate="false" warningaserror="false">
  <wiringdesign name="Design1" revision="1" releasestatus="Complete"/>
  <wiringdesign name="Design2" revision="1" releasestatus="Complete"/>
</project>
```

Below is the explanation of each attribute in configuration xml file

projectname: Name of the project under consideration.

annotate: Defines if the diagrams need to be annotated after the release of the design. Accepts true/false.

warningaserror: Defines if the warnings that arise out of design rule checks to be treated as errors. Accepts true/false.

name: Name of the design to be released.

revision: Revision of the design to be released.

shortdesc: Short description of the design to be released.

partnumber: Part number of the harness design to be released.

releasestatus Target release level of type Released to which the design needs to be transitioned.

The following is a sample result:

```
Mon Nov 18 15:38:50 IST 2013 -- Started execution of the task ReleaseDesignsTask
Task - Release Designs
Task Execution ID - UID3c9e58-1426aaebc2f-2d74541398def4af87da30952f02f321
Task Parameter File - E:\chs_home\temp\ReleaseDesignsTask_1426aaebc2f.xml
User - system
PROGRESS - Start Phase Releasing Design: Design1:1 ---- (1)
OUTPUT - Releasing Design: Design1:1
OUTPUT - Release Designs summary
   Design   Status   Comments
   Design1:1   Succeeded   Design transitioned to "Complete" release level.
OUTPUT - Writing DRC results to "E:\chs_home\temp\UID3c9e58-1426aaebc2f-
2d74541398def4af87da30952f02f321.csv"
PROGRESS - Phase - 1 Step ---- (1/1)
Mon Nov 18 15:38:53 IST 2013 -- Finished execution of the task ReleaseDesignsTask
```

Using the BuildLists list service

Name	ListBuildLists
Request schema	$CAPITAL_HOME/dtd/bridgesListBuildListsRequest.xsd
Response schema	$CAPITAL_HOME/dtd/bridgesListBuildListsResponse.xsd
Response	List of Build Lists meta data

See the provided ListBuildListsClient example Java class for how to invoke this service and how to process the response.

```
<?xml version="1.0" encoding="UTF-8"?>
<project name='testProject'>
```

```
        <buildlisttype>logic</buildlisttype>
        <buildlisttype>harness</buildlisttype>
</project>
```

The response payload provides the list of Build Lists as children of the root element.

Following is a sample response showing all the metadata information of each BuildList.

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<buildlists xmlns="http://www.mentor.com/harness/Schema/bridgesListBuildListsResponse"
author="system" creationDate="2014-07-31" creationTime="18:03:05" schemaVersion="1_0">
  <buildlist buildlistmaintenance="MANUAL" description="" effectivityvalue="" folderpath=""
id="UID4e2806-1476cdbac10-2d74541398def4af87da30952f02f321" modified="6" name="logicBuildList"
releaselevel="Draft" type="logic"/>
  <buildlist buildlistmaintenance="MANUAL" description="logic build list 2" effectivityvalue=""
folderpath="f1" id="UIDaee210-1478bfa9601-2d74541398def4af87da30952f02f321" modified="3"
name="logicBuildList2" releaselevel="Draft" type="logic"/>
</buildlists>
```

If the requested project belongs to a domain which is not assigned to the Capital user used to authenticate the request, then this web service returns a SOAP fault.

```
SOAP Fault = Project with specified attributes cannot be found or access not granted to <user>
```

Using the Build List Importer service

Name	ImportBuildLists
Request schema	$CAPITAL_HOME/dtd/bridgesBuildListsExportRequest.xsd
Response schema	$CAPITAL_HOME/dtd/bridgesimportbuildlistsresponse.xsd
Response	XML providing the list of failed Build Lists

See the provided ImportBuildListsClient example Java class for how to invoke this service and how to process the response.

The following options are provided as attributes to the project element

- overwritebuildlists – default value is “false”. By default import of existing Build Lists will be skipped. If this parameter is specified with true value, then the existing Build Lists in the project will be overwritten provided the existing Build List release level allows editing.

The import of Build Lists is atomic in nature. If one of the BuildLists fail to import, none of the BuildLists in the request will be imported.

The request payload contains the Build Lists to import as per the XML schema. The root element identifies the project into which this information is to be imported.

Following is a sample request to create a Build List

```
<?xml version="1.0" encoding="UTF-8"?>
<project name="Sample">
  <buildlistmgr id="_1">
```

```
    <buildlist buildlistmaintenance="MANUAL" description="" effectivityvalue="" 
folderpath="Sample\Sample1" id="_2" name="HarnBuildList" releaselevel="Draft" type="harness">
      <buildlistdetail design="UIDc4ee53-147431320b3-d24e0123c53c2f7d4a85ff466f43f062" id="_2"/>
      <ecoref buildlist_id="_2" id="UID4a045c-133b6e9569f-0ccf12edafedf30717990c6b7fe0c1bc"/>
      <property name="prop1" stability="editable" type="String" val="propvalue"/>
    </buildlist>
  </buildlistmgr>
</project>
```

Following is a sample request to import the Build List in overwrite mode.

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<project xmlns="http://www.mentor.com/harness/Schema/bridgesbuildList" name="example4" 
overwritebuildlists="true">
  <buildlistmgr id="_1">
    <buildlist buildlistmaintenance="MANUAL" description="" effectivityvalue="" folderpath="" 
id="_2" name="logicBuildList" releaselevel="Draft" type="logic">
      <buildlistdetail design="UID4a045c-133b6e956a4-0ccf12edafedf30717990c6b7fe0c1bc" id="_2"/>
      <ecoref buildlist_id="_2" id="UID4a045c-133b6e9569a-0ccf12edafedf30717990c6b7fe0c1bc"/>
    </buildlist>
  </buildlistmgr>
</project>
```

Following is a sample response payload in case of a successful import of all the Build Lists in the request.

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<project xmlns="http://www.mentor.com/harness/Schema/bridgesimportbuildlistsresponse" 
author="system" creationDate="2014-07-31" creationTime="16:13:02" dtdversion="BuildListMgr 1.0" 
name="example4" schemaVersion="1_0"/>
```

Following is a sample response in case of a failure during import.

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<project xmlns="http://www.mentor.com/harness/Schema/bridgesimportbuildlistsresponse" 
author="system" creationDate="2014-07-31" creationTime="16:13:02" dtdversion="BuildListMgr 1.0" 
name="example4" overwritebuildlists="true" schemaVersion="1_0">
    <buildlist error="Build List logicBuildList : release level  Draft2 is not available." 
name="logicBuildList" type="logic"/>
</project>
```

If the requested project belongs to a domain which is not assigned to the Capital user used to authenticate the request, then this web service returns a SOAP fault.

```
SOAP Fault = Project with specified attributes cannot be found or access not granted to <user>
```

Using the Build List exporter service

Name	ExportBuildListService
Request schema	$CAPITAL_HOME/dtd/bridgesBuildListsExportRequest.xsd
Response schema	$CAPITAL_HOME/dtd/bridgesbuildList.xsd
Response	Capital-XML representation of BuildList

See the provided ExportBuildListClient example Java class for how to invoke this service and how to process the response.

The request payload needs to include the root element as per the XML schema, with identification of the project by name.
The following options are provided as attributes to the Build List element

- asattachment – default value is “true”. The BuildList is exported as gzip attachment by default. If this parameter is specified with false value, then the response XML will be exported as xml file.
- name – Name of the BuildList that needs to be exported.
- type – Type of the BuildList that needs to be exported. E.g., logic/harness/analysis/topology.
- id – ID of the BuildList that needs to be exported.

Sample request:

```
<project name="example4">
<buildlist asattachment="false" name="logicBuildList" type="logic"/>
</project>
```

Following is a sample response payload that contains details of the exported BuildList.

```
<?xml version="1.0" encoding="UTF-8"?><project
xmlns="http://www.mentor.com/harness/Schema/bridgesbuildList" author="system" creationDate="2014-
07-31" creationTime="16:24:03" dtdversion="BuildListMgr 1.0" name="example4" schemaVersion="1_0">
     <buildlistmgr haschildren="true" id="UID4a045c-133b6e9567b-0ccf12edafedf30717990c6b7fe0c1bc"
    modified="7">
      <buildlist buildlistmaintenance="MANUAL" description="" effectivityvalue="" folderpath="" id="UID4e2806-1476cdbac10-2d74541398def4af87da30952f02f321" modified="4"
    name="logicBuildList" releaselevel="Draft" type="logic">
        <buildlistdetail design="UID4a045c-133b6e956a4-0ccf12edafedf30717990c6b7fe0c1bc"
    id="UID4e2806-1476cdbac10-2d74541398def4af87da30952f02f321"/>
      <ecoref buildlist id="UID4e2806-1476cdbac10-2d74541398def4af87da30952f02f321"
id="UID4a045c-133b6e9569a-0ccf12edafedf30717990c6b7fe0c1bc"/>
      <ecoref buildlist_id="UID4e2806-1476cdbac10-2d74541398def4af87da30952f02f321"
id="UID4a045c-133b6e9569f-0ccf12edafedf30717990c6b7fe0c1bc"/>
    </buildlist>
  </buildlistmgr>
</project>
```

If the requested project belongs to a domain which is not assigned to the Capital user used to authenticate the request, then this web service returns a SOAP fault.

```
SOAP Fault = Project with specified attributes cannot be found or access not granted to <user>
```

Using the Batch Bridge In Task service

Name	BatchBridgeInTask
Request schema	$CAPITAL_HOME/dtd/bridgessubmittask.xsd
Response schema	-
Response	-

See the provided BatchBridgeInTaskClient example Java class for how to invoke this service and how to process the response.

A BatchBridgeInTask may be submitted using the *Submit Task* service.

The request payload needs to include the root element as per the XML schema, the following options are specified as attributes to the root element (if unspecified, the default value is *true*):

- *cron_expression*: The cron expression is used to describe the schedule for running the task. If you don't specify anything then task will be executed as soon as possible. If you require assistance in the construction of a cron expression, then there are several good resources available on the internet, eg *www.cronmaker.com*.
    - Example cron expression to run a task at specified time say 4.20 PM, 24th October 2012 is "00 20 16 24 10 ? 2012"
    - Example cron expression to run task 10 PM every Wednesday and Saturday is "0 0 22 ? * WED,SAT *"
    - Example cron expression to run task 12 PM every day is "0 0 12 1/1 * ? *"
- *instance_name*: The name that is to be given to this submitted instance of the task.
- *name*: This is the name of the Capital task i.e. BatchBridgeInTask.
- *Id*: This is an optional attribute and if provided an existing task with this UID will be modified.
- *application* : This is an optional attribute and can be used to specify the application using which task needs to be run. Default value of this attribute is CapitalHarnessXC.exe. The other applicable value for this task is CapitalModularXC.exe.

```
<taskspec cron_expression="" instance_name="BridgeFEMTask" name="BatchBridgeInTask"/>
```

Input for BatchBridgeInTask is provided as 3 different XML SOAP attachments. Content-ID and Content-type for each of them are as per the following table:

S.No.	SOAP attachment description	Content-Type	Content-ID
1	Task parameters depicting key details like Project, Design, Adaptor etc.	application/xml	FEMParams
2	Bridge configuration file depicting various Bridge settings for Bridge In	application/xml	BRIDGE_CONFIG_FILE
3	Source adaptor file to be Bridged In	application/xml	BRIDGE_IMPORT_FILE

A sample FEMParams file that could be used is available here:
$CAPITAL_HOME\doc\webservice\data\BatchBridgeInTask_paramWithAttachments.xml

A sample BRIDGE_CONFIG_FILE is available here:
$CAPITAL_HOME\doc\webservice\data\BridgesTestConfigurations.xml

It is recommended that the required BRIDGE_CONFIG_FILE be automatically generated (by dumping from the existing Bridge Settings in the design tool). This can be done by building the sample Extensibility plugin "ExportBridgeConfigurationsAction" and running it from Capital Harness XC or Capital ModularXC. The sample plugin code is available here: $CAPITAL_HOME\doc\plugin\examples\Java\src\com\example\plugin\bridge

Once the BatchBridgeInTask is complete, the Bridge adaptor report and the Change Manager / Bridge In report are saved as task attachments.

S.No.	BatchBridgeInTask artifact	Task attachment name
1	Bridge adaptor html report	CMAdaptorReport
2	Bridge In / Change Manager csv report	ChangeManagerReport

The List Tasks service may be used to list all the BatchBridgeInTask instances of a given state. The Describe Task service may be used to get the logs and attachments of any BatchBridgeInTask.

External Web Services deployed in CIS

The *embedded* attribute, if set true, specifies that this web server will run within the Capital Manager process (and hence, starting the Capital Manager server will also start this server). This option is currently available only for the Capital Enterprise Reporting Server.

Notes:

- This configuration is necessary only to avoid conflicts with existing processes already listening on the port number.
- If not configured, the default port number is 49901
- this manual only covers CIS web services: these are the services exposed by Capital to third parties. The other web servers provide internal web services, not for use outside of Capital

How to invoke a CIS web service

The included Java examples provide all documentation necessary to invoke any CIS web service.

They are all based on a common client abstract class (com.example.webservice.client.AbstractClient) which provides the main generic behavior:

- Build the request SOAP message, with attachments if required
- Send the request message to CIS using a URL reference
- Process the response:
    - If a response is expected and is actually a fault, raise an exception
    - If no response is expected and one is actually received, raise an exception
    - Otherwise, if a response is expected, process its payload, and attachments if present

Each web service example client class extends this base class to provide the specific data (service name, request XML payload, etc…) and behavior (processing the response) by overriding a small number of base class methods.

How to build an example Java client

You need the ant tool installed to build the Java samples with the build.xml file provided (otherwise, you can use your usual Java build process).

- From your shell command line, go to the $CAPITAL_HOME/doc/webservice/examples/client directory.
- Run “ant”: this will use the provided build.xml file to compile the Java classes, create the wsclient.jar and wscommon.jar jar files and copy them to the created $CAPITAL_HOME/WebServiceExamples directory. It will also copy the provided run.bat and run.sh scripts to the same directory.

How to run an example Java client

Use the provided scripts run.bat (DOS) or run.sh (UNIX): Go to the $CAPITAL_HOME/WebServiceExamples directory and invoke the example client of your choice by passing the name of the class as a parameter to the script as follows:

- run.sh ListDesignsClient

It is also recommended to import the provided datasets from the $CAPITAL_HOME/doc/webservice/data directory, as this data has been used to illustrate the examples in this document.

Using the Projects list service

Name	ListProjects
Request schema	$CAPITAL_HOME/dtd/bridgesprojects.xsd
Response schema	$CAPITAL_HOME/dtd/bridgesprojects.xsd
Response	List of projects meta data

See the provided ListProjectsClient example Java class for how to invoke this service and how to process the response.

The request payload needs only include the root element as per the XML schema:

```
<projects/>
```

The response payload uses the same schema and inserts the list of projects as children to the root element. Each project element provides a *name*, *description* and *id* (the unique identifier of this element in the Capital database) attribute.

```
<projects xsi:schemaLocation="http://www.mentor.com/harness/Schema/bridgesprojects
H:\capital home/dtd/bridgesprojects.xsd"
 author="system" creationDate="2007-11-07" creationTime="08:22:36" schemaVersion="1_0"
xmlns="http://www.mentor.com/harness/Schema/bridgesprojects">
  <project id="UIDabf9e6-1152dc8e716-f528764d624db129b32c21fbca0cb8d6" name="example1"
description="Web Services documentation \nProject example 1"/>
  <project id="UIDabf9e6-1152dc8e7b1-f528764d624db129b32c21fbca0cb8d6" name="example2"
description="Web Services documentation \nProject example 2"/>
</projects>
```

This web service returns only the projects with no domain or which domain includes the Capital user used to authenticate the request.

Using the Designs list service

Name	ListDesigns
Request schema	$CAPITAL_HOME/dtd/bridgesdeviceusage.xsd
Response schema	$CAPITAL_HOME/dtd/bridgesdesigns.xsd
Response	List of designs meta data

See the provided ListDesignsClient example Java class for how to invoke this service and how to process the response.

The request payload needs to include the root element as per the XML schema, with identification of the project, by either *name* or *id*. Note that only the root element for this schema is used in this request – and is actually the same as other equivalent schemas bridgesobjecttypeinfo.xsd and bridgesoptions.xsd.
The parent design element under harness design types indicates the parent design id of the derivative design.

```
<project name="example1"/>
```

The response payload uses a different schema and provides the list of designs as children of the root element. Each design element provides a *name*, *short description*, *revision*, *type* and *id* attribute.

```
<designs xsi:schemaLocation="http://www.mentor.com/harness/Schema/bridgesdesigns
H:\capital_home/dtd/bridgesdesigns.xsd" author="system" creationDate="2007-11-07"
creationTime="09:06:18" schemaVersion="1 0"
xmlns="http://www.mentor.com/harness/Schema/bridgesdesigns">
  <design id="UIDe69c0a-1152dcc7554-f528764d624db129b32c21fbca0cb8d6" name="Design1" revision="1"
shortdesc="Logic design example 1" designtype="logicdesign" OptionExpression=" IllumEntry
&amp;&amp; LHD || IllumEntry &amp;&amp; RHD" ReleaseLevel="Draft" DesignAbstraction="System
schematic" DomainName="public" CreatedBy="System" CreatedOn="2007-11-07" ModifiedBy="system"
ModifiedOn="2007-11-07" />
  <design id="UIDe69c0a-1152dcc76eb-f528764d624db129b32c21fbca0cb8d6" name="IntegratorDesign1"
revision="1" shortdesc="Integrator design example 1" designtype="topologydesign"
OptionExpression=" IllumEntry &amp;&amp; LHD || IllumEntry &amp;&amp; RHD" ReleaseLevel="Draft"
DesignAbstraction="wiring" DomainName="public" CreatedBy="System" CreatedOn="2007-11-07"
ModifiedBy="system" ModifiedOn="2007-11-07"/>
  <design id="UIDe69c0a-1152dcc7876-f528764d624db129b32c21fbca0cb8d6" name="HarnessDesign1"
revision="1" shortdesc="Harness design example 1" designtype="harnessdesign"
PartNumber="harnesscustomer" HarnessGroup="HSGrp" MCadID="X7EEE52" Sealed="false" Plugged="false"
OptionExpression=" IllumEntry &amp;&amp; LHD || IllumEntry &amp;&amp; RHD" ReleaseLevel="Draft"
DesignAbstraction="Harness" DomainName="public" CreatedBy="System" CreatedOn="2007-11-07"
ModifiedBy="system" ModifiedOn="2007-11-07">
< parentdesign id=" UIDba4871-1422e29449e-279e2bee43fed9b5aeb17e7eb27cb620" />
</design>
</designs>
```

If the requested project belongs to a domain which is not assigned to the Capital user used to authenticate the request, then this web service returns a SOAP fault.

```
SOAP Fault = Project with specified attributes cannot be found or access not granted to <user>
```

Using the Diagrams list service

Name	ListDesignDiagrams
Request schema	$CAPITAL_HOME/dtd/bridgesSVGdiagrams.xsd
Response schema	-
Response	List of diagrams meta data

See the provided ListDiagramsClient example Java class for how to invoke this service and how to process the response.

The request payload needs to include the root element as per the XML schema, with identification of the parent design: sufficient attributes must be provided to uniquely identify it. For instance, if only the *name* (and *projectname*) is provided and it matches more than one design, then a SOAP fault is returned. If the *id* attribute is provided, then no other attribute needs to be provided as it uniquely identifies the design. Note also that the design is uniquely identified by the (*name*, *revision*, *short description*) triplet within a given project.
Despite the name of the root element (*wiringdesign*), all types of designs are supported (Logic, Integrator or Harness designs).

```
<wiringdesign name="Design1" projectname="example1"/>
```

The response payload uses the same schema and inserts the list of diagrams as children of the root element. Each diagram element provides a name, type and id attribute.

```
<wiringdesign xsi:schemaLocation="http://www.mentor.com/harness/Schema/bridgesSVGdiagrams
H:\capital_home/dtd/bridgesSVGdiagrams.xsd" author="system" creationDate="2007-11-07"
creationTime="14:09:24" schemaVersion="1 0" id="UIDe69c0a-1152dcc7554-
f528764d624db129b32c21fbca0cb8d6" name="Design1" projectid="UIDabf9e6-1152dc8e716-
f528764d624db129b32c21fbca0cb8d6" projectname="example1" revision="1" shortdesc="Logic design
example 1" designtype="logicdesign"
xmlns="http://www.mentor.com/harness/Schema/bridgesSVGdiagrams">
  <diagram type="wiring" id="UIDe69c0a-1152dcc767d-f528764d624db129b32c21fbca0cb8d6"
name="Diagram1"/>
  <diagram type="wiring" id="UIDd03176-1152e6e4096-f528764d624db129b32c21fbca0cb8d6"
name="Diagram2"/>
</wiringdesign>
```

If the requested design project belongs to a domain which is not assigned to the Capital user used to authenticate the request, then this web service returns a SOAP fault.

```
SOAP Fault = Design with specified attributes cannot be found in accessible domains.
```

Using the SVG diagram generator service

Name	SVGDiagrams
Request schema	$CAPITAL_HOME/dtd/bridgesSVGdiagrams.xsd
Response schema	http://www.w3.org/2000/svg
Response	SVG representation of the diagram

See the provided RetrieveSVGDiagramClient example Java class for how to invoke this service and how to process the response.

The request payload needs to include the root element as per the XML schema. The request should allow to identify a unique diagram in the following way:

- By identifying the parent design in a unique way (as in Using the Diagrams list service). In this case, a *diagram* child element can be provided to specify the diagram (by *name*). If none is specified, the first one found is selected (this can be unpredictable if the design has many diagrams). If more than one is specified, only the first one is selected.

```
<wiringdesign name="Design1" projectname="example1">
   <diagram name="Diagram1"/>
</wiringdesign>
```

- By identifying the diagram by its id attribute. In this case, the parent design needs no attribute to specify it.

```
<wiringdesign>
    <diagram id='UIDe69c0a-1152dcc767d-f528764d624db129b32c21fbca0cb8d6'/>
</wiringdesign>
```

Additional attributes can be specified for each diagram explicitly included in the request to control the default resolution of rendition. Those attributes (*dpi, maxwidth, maxheight*) are fully documented in the schema file.

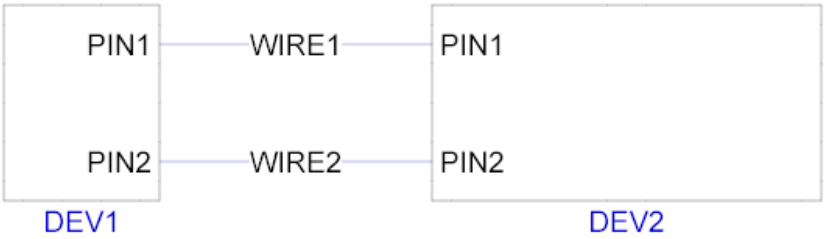
An attribute “generateLegacySVG” can also be specified for diagram.

- User may send this flag in Web Service request to receive desired SVG
  - FALSE-Lean SVG will be returned as response.
  - TRUE-Legacy SVG will be returned as response.
- If user does not send this flag in Web service request, then SVG will be returned as per the System Preference
- Please note: By default System Preference will be set to “False” i.e. Lean SVG.

The web service checks that the diagram design is at a release level which includes the SVG service behavior. Otherwise, a SOAP fault is returned:

```
SOAP Fault = SVG diagram export operation is not allowed for design <design name> at release
level <release level name>.
```

The response payload uses the published SVG schema and can directly be viewed in a SVG browser.



If the requested diagram project belongs to a domain which is not assigned to the Capital user used to authenticate the request, then this web service returns a SOAP fault.

```
SOAP Fault = Diagram with specified attributes cannot be found in accessible domains.
```

This service has been enhanced to export the SVG diagram as SOAP attachment. The attachment will be in gzip format and can be processed on the client side. For exporting it as an attachment, an optional parameter is provided (“asattachment”). Default value for this parameter will be false.

```
<wiringdesign name="Design1" projectname="example1" asattachment=”true”>
```

Using the DXF diagram generator service

Name	DXFDiagrams
Request schema	$CAPITAL_HOME/dtd/bridgesDXFdiagrams.xsd
Response schema	-
Response	DXF representations of diagrams

See the provided RetrieveDXFDiagramClient example Java class for how to invoke this service and how to process the response.

The request payload needs to include the root element as per the XML schema. The request should allow to identify the diagrams for which the DXF representation is needed. Contrarily to the previous SVG diagram generator service, the DXF service allows to output several diagrams in the same response payload, as long as they belong to the same design. Hence if no diagram is specified, all diagrams in the parent design will be output.
The DXF web service has been enhanced to export the diagrams as SOAP attachments. With the addition of two new parameters

(“asattachment”,”asprintregions”), this service is capable of exporting all the print regions in the diagram. Please note that all print regions (active and Inactive) will be exported.

```
<wiringdesign name="Design1" projectname="example1" asattachment=”true” asprintregions=”true”/>
```

In the absence of these new attributes, the request root element looks like this:

```
<wiringdesign name="Design1" projectname="example1"/>
```

The web service checks that the diagram design is at a release level which includes the SVG service behavior. Otherwise, a SOAP fault is returned:

```
SOAP Fault = SVG diagram export operation is not allowed for design <design name> at release
level <release level name>.
```

The response payload is based on the same schema and the optional attributes. In the absence of these attributes (“asattachment”,”asprintregions”) the response payload includes the DXF representation for each requested diagram as a CDATA section belonging to the *diagram* elements. The content of each CDATA section can be extracted and persisted as a separate DXF file by the client program.
If the attributes (“asattachment”,”asprintregions”) are enabled, then the DXF files will be exported as an attachment and the client program has to process the SOAP attachments to get the files. The attachment header will contain the following information which can be used on the client side:
Content-ID → <Name of Diagram>_<Print Region Name>
printregionstatus → Active/InActive
Content-type → attachment/gzip.

If the requested diagram project belongs to a domain which is not assigned to the Capital user used to authenticate the request, then this web service returns a SOAP fault.

```
SOAP Fault = Diagram with specified attributes cannot be found in accessible domains.
```

Using the SVG symbol generator service

Name	SVGSymbol
Request schema	$CAPITAL_HOME/dtd/bridgesSVGsymbol.xsd
Response schema	http://www.w3.org/2000/svg
Response	SVG representation of the symbol

See the provided RetrieveSVGSymbolClient example Java class for how to invoke this service and how to process the response.

This service allows to generate the SVG representation of a symbol or a border authored in Capital Symbol.

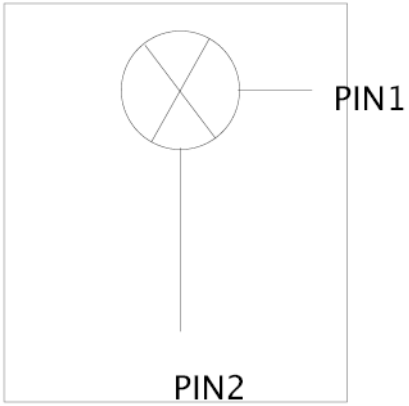
The request payload needs to include the root element as per the XML schema with sufficient attributes to uniquely identify the symbol or border. For instance, only the *id* attribute can provided, or otherwise the parent library name (*libname*) and the symbol or border name (*name*).

An attribute “generateLegacySVG” can also be specified for symbol.

- User may send this flag in Web Service request to receive desired SVG
  - FALSE-Lean SVG will be returned as response.
  - TRUE-Legacy SVG will be returned as response.
- If user does not send this flag in Web service request, then SVG will be returned as per the System Preference
- Please note: By default System Preference will be set to “False” i.e. Lean SVG.

```
<symbol libname='symbollib' name='symb1'/>
```

The response payload uses the published SVG schema and can directly be viewed in a SVG browser.



Using the Design exporter service

Name	ExportProjectDesign
Request schema	$CAPITAL_HOME/dtd/bridgesdesignstatus.xsd
Response schema	$CAPITAL_HOME/dtd/project.dtd
Response	Capital-XML representation of the design

See the provided ExportDesignClient and ExportDesignAsAttachmentClient example Java classes for how to invoke this service and how to process the response.

The request payload needs to include the root element as per the XML schema, with identification of the design: sufficient attributes must be provided to uniquely identify it. For instance, if only the *name* (and *projectname*) is provided and it matches more than one design, then a SOAP fault is returned. If the *id* attribute is provided, then no other attribute needs to be provided as it uniquely identifies the design. Note also that the design is uniquely identified by the (*name*, *revision*, *short description*) triplet within a given project.
Despite the name of the root element (*wiringdesign*), all types of designs are supported (Logic, Integrator or Harness designs).
The *includeassociateddesign* attribute is taken into account for Integrator designs and Harness Designs. If the value *true* (which is the default) is specified, then all associated Logic designs for Integrator design and derivative designs for Harness Design will be included.
The *includegeneratedwiring* attribute is taken into account for Integrator designs only. If the value true (which is the default) is specified, then generated wiring is also exported.

```
<wiringdesign name="Design1" projectname="example1"/>
```

From version 2009.1 and onwards, this service supports two response formats, based on the new request optional attribute *asattachment*: if this attribute is specifed as true, then the exported design XML is provided as a gzip attachment and the content is identical as if the design had been exported from the Capital Project interactive function (including <!DOCTYPE> declaration.
Otherwise, the “old” format is provided, i.e. the response payload is directly included in the SOAP body and is based on the Capital project DTD from which the <!DOCTYPE> declaration has been removed for SOAP compatibility.
Therefore, an extra attribute *dtdversion* is added to the root *project* element to indicate the DTD version. This allows to correctly upgrade the XML when re-importing the design in a higher level of Capital.

```
<project baseid="UIDabf9e6-1152dc8e716-f528764d624db129b32c21fbca0cb8d6" description="Web
Services documentation \nProject example 1" domainref="" dtdversion="1.48" id="_1" modified="8"
name="example1">
…
```

Note: in order to re-import the “old” format through the Capital Project interactive function, a DTD declaration must be inserted just before the root element:

```
<!DOCTYPE project PUBLIC "-//Mentor Graphics Corporation//Project [version]//EN"
"[$CAPITAL_HOME]/dtd/project.dtd">
```

Where the [$CAPITAL_HOME] field above must be substituted with the path of your Capital installation directory and the [version] field by the value of the *dtdversion* attribute.

If the requested design project belongs to a domain which is not assigned to the Capital user used to authenticate the request, then this web service returns a SOAP fault.

```
SOAP Fault = Design with specified attributes cannot be found in accessible domains.
```

Using the Option information exporter service

Name	ExportOptions
Request schema	$CAPITAL_HOME/dtd/bridgesoptions.xsd
Response schema	$CAPITAL_HOME/dtd/bridgesoptions.xsd
Response	Capital-XML representation of Option information

See the provided ExportOptionsClient example Java class for how to invoke this service and how to process the response.

The request payload needs to include the root element as per the XML schema, with identification of the project by name. The root element can be sent with no attribute in which case System option definitions will be output.

```
<project name="example1"/>
```

The response payload contains all option definitions, including configuration, families and sets.

```
<project xmlns="http://www.mentor.com/harness/Schema/bridgesoptions"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" author="system" creationDate="2007-11-08"
creationTime="22:39:44" name="example1" schemaVersion="1 0"
xsi:schemaLocation="http://www.mentor.com/harness/Schema/bridgesoptions
H:\capital_home/dtd/bridgesoptions.xsd">
<optionmgr baseid=" 1" id=" 1" modified="9" type="options">
        <option description="option 1" id="_2" isvariant="false" name="opt1"/>
    <option description="option 2" id="_3" isvariant="false" name="opt2"/>
    <option description="option 3" id="_4" isvariant="true" name="opt3"/>
    <option description="option 4" id="_5" isvariant="false" name="opt4"/>
    <inclusiveoptions optionref=" 5">
               <refedoption optionref="_2"/>
               <refedoption optionref="_3"/>
    </inclusiveoptions>
    <optionconfiguration baseid="_6" description="configuration 1" id="_6" name="conf1">
               <refedoption optionref="_2"/>
```

```
                <refedoption optionref="_3"/>
        </optionconfiguration>
    <optionconfiguration baseid="_7" description="configuration 2" id="_7" name="conf2">
                <refedoption optionref=" 4"/>
                <refedoptionconfiguration optionconfigurationref="_6"/>
        </optionconfiguration>
    <optionfamily baseid="_8" id="_8" name="family1">
                <refedoption optionref="_2"/>
                <refedoption optionref=" 3"/>
                <optionset baseid="_9" id="_9" name="set1">
                        <refedoption optionref="_2"/>
                        <refedoption optionref=" 3"/>
                </optionset>
        </optionfamily>
</optionmgr>
</project>
```

If the requested project belongs to a domain which is not assigned to the Capital user used to authenticate the request, then this web service returns a SOAP fault.

```
SOAP Fault = Project with specified attributes cannot be found or access not granted to <user>
```

Note: in order to re-import this data through the Capital Project interactive function, a DTD declaration must be inserted just before the root element:

```
<!DOCTYPE optionmgr PUBLIC "-//Mentor Graphics Corporation//OptionMgr 1.0//EN"
"[$CAPITAL_HOME]/dtd/options.dtd">
```

Where the [$CAPITAL_HOME] field above must be substituted with the path of your Capital installation directory.

Using the Object Type information exporter service

Name	ExportObjectTypeInfo
Request schema	$CAPITAL_HOME/dtd/bridgesobjecttypeinfo.xsd
Response schema	$CAPITAL_HOME/dtd/bridgesobjecttypeinfo.xsd
Response	Capital-XML representation of Object Type information

See the provided ExportObjectTypeInfoClient example Java class for how to invoke this service and how to process the response.

The request payload needs to include the root element as per the XML schema, with identification of the project by name. The root element can be sent with no attribute in which case System Object Type information will be output.

```
<project name="example1"/>
```

The response payload contains all Object Type information, including names, properties and ranges.

```
<project xmlns="http://www.mentor.com/harness/Schema/bridgesobjecttypeinfo"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" author="system" creationDate="2007-11-08"
creationTime="22:59:08" id="UIDabf9e6-1152dc8e716-f528764d624db129b32c21fbca0cb8d6"
```

```
name="example1" schemaVersion="1_0"
xsi:schemaLocation="http://www.mentor.com/harness/Schema/bridgesobjecttypeinfo
H:\capital_home/dtd/bridgesobjecttypeinfo.xsd">
  <objecttypeinfo id="UIDabf9e6-1152dc8e781-f528764d624db129b32c21fbca0cb8d6" name="BACKSHELL"
type="normal"/>
  <objecttypeinfo id="UIDabf9e6-1152dc8e789-f528764d624db129b32c21fbca0cb8d6" name="GROUND"
type="normal"/>
  <objecttypeinfo id="UIDabf9e6-1152dc8e782-f528764d624db129b32c21fbca0cb8d6" name="TERMINATION"
type="normal"/>
  <objecttypeinfo id="UIDabf9e6-1152dc8e783-f528764d624db129b32c21fbca0cb8d6" name="GENERAL"
type="general"/>
  <objecttypeinfo id="UIDabf9e6-1152dc8e784-f528764d624db129b32c21fbca0cb8d6" name="NETCONDUCTOR"
type="normal"/>
  <objecttypeinfo id="UIDabf9e6-1152dc8e785-f528764d624db129b32c21fbca0cb8d6"
name="WIRECONDUCTOR" type="normal"/>
  <objecttypeinfo id="UIDabf9e6-1152dc8e786-f528764d624db129b32c21fbca0cb8d6"
name="SHIELDCONDUCTOR" type="normal"/>
  <objecttypeinfo id="UIDabf9e6-1152dc8e787-f528764d624db129b32c21fbca0cb8d6" name="CONNECTOR"
type="normal"/>
  <objecttypeinfo id="UIDabf9e6-1152dc8e788-f528764d624db129b32c21fbca0cb8d6" name="PIN"
type="normal"/>
  <objecttypeinfo id="UIDabf9e6-1152dc8e78a-f528764d624db129b32c21fbca0cb8d6" name="DEVICE"
type="normal">
        <propertytemplate id="UID66ae42-116217a0fc9-f528764d624db129b32c21fbca0cb8d6" name="prop1"
type="String"/>
        <propertytemplate id="UID66ae42-116217a0fca-f528764d624db129b32c21fbca0cb8d6" name="prop2"
type="Integer">
                <range baseid="UID66ae42-116217a0fcc-f528764d624db129b32c21fbca0cb8d6"
id="UID66ae42-116217a0fcc-f528764d624db129b32c21fbca0cb8d6" startvalue="1" type="Integer"/>
                <range baseid="UID66ae42-116217a0fcd-f528764d624db129b32c21fbca0cb8d6"
id="UID66ae42-116217a0fcd-f528764d624db129b32c21fbca0cb8d6" startvalue="2" type="Integer"/>
                <range baseid="UID66ae42-116217a0fce-f528764d624db129b32c21fbca0cb8d6"
id="UID66ae42-116217a0fce-f528764d624db129b32c21fbca0cb8d6" startvalue="3" type="Integer"/>
        </propertytemplate>
    <propertytemplate id="UID66ae42-116217a0fcb-f528764d624db129b32c21fbca0cb8d6" name="prop3"
type="String">
                <range baseid="UID66ae42-116217a0fcf-f528764d624db129b32c21fbca0cb8d6"
id="UID66ae42-116217a0fcf-f528764d624db129b32c21fbca0cb8d6" startvalue="a" type="String"/>
                <range baseid="UID66ae42-116217a0fd0-f528764d624db129b32c21fbca0cb8d6"
id="UID66ae42-116217a0fd0-f528764d624db129b32c21fbca0cb8d6" startvalue="b" type="String"/>
        </propertytemplate>
    <nametemplate description="device 1" id="UID66ae42-116217a0fc7-
f528764d624db129b32c21fbca0cb8d6" name="DEV1" shortdescription="dev 1"/>
        <nametemplate description="device 2" id="UID66ae42-116217a0fc8-
f528764d624db129b32c21fbca0cb8d6" name="DEV2" shortdescription="dev 2"/>
  </objecttypeinfo>
  <objecttypeinfo id="UIDabf9e6-1152dc8e78b-f528764d624db129b32c21fbca0cb8d6" name="MULTICORE"
type="normal"/>
  <objecttypeinfo id="UIDabf9e6-1152dc8e78c-f528764d624db129b32c21fbca0cb8d6" name="SPLICE"
type="normal"/>
  <objecttypeinfo id="UIDabf9e6-1152dc8e78d-f528764d624db129b32c21fbca0cb8d6" name="ASSEMBLY"
type="normal"/>
  <objecttypeinfo id="UIDabf9e6-1152dc8e78e-f528764d624db129b32c21fbca0cb8d6" name="PROJECT"
type="normal"/>
  <objecttypeinfo id="UIDabf9e6-1152dc8e78f-f528764d624db129b32c21fbca0cb8d6" name="LOGIC-DESIGN"
type="normal"/>
  <objecttypeinfo id="UIDabf9e6-1152dc8e790-f528764d624db129b32c21fbca0cb8d6" name="INTEGRATOR-
DESIGN" type="normal"/>
  <objecttypeinfo id="UIDabf9e6-1152dc8e791-f528764d624db129b32c21fbca0cb8d6" name="LOGIC-
DIAGRAM" type="normal"/>
  <objecttypeinfo id="UIDabf9e6-1152dc8e792-f528764d624db129b32c21fbca0cb8d6" name="PLANE-DESIGN"
type="normal"/>
  <objecttypeinfo id="UIDabf9e6-1152dc8e793-f528764d624db129b32c21fbca0cb8d6" name="HARNESS-
DIAGRAM" type="normal"/>
  <objecttypeinfo id="UIDabf9e6-1152dc8e794-f528764d624db129b32c21fbca0cb8d6" name="HARNESS-
DESIGN" type="normal"/>
  <objecttypeinfo id="UIDabf9e6-1152dc8e795-f528764d624db129b32c21fbca0cb8d6" name="BUNDLE"
type="normal"/>
  <objecttypeinfo id="UIDabf9e6-1152dc8e796-f528764d624db129b32c21fbca0cb8d6" name="CLIP"
type="normal"/>
  <objecttypeinfo id="UIDabf9e6-1152dc8e797-f528764d624db129b32c21fbca0cb8d6" name="GROMMET"
type="normal"/>
  <objecttypeinfo id="UIDabf9e6-1152dc8e798-f528764d624db129b32c21fbca0cb8d6" name="BUNDLE INS"
type="normal"/>
```

```
  <objecttypeinfo id="UIDabf9e6-1152dc8e799-f528764d624db129b32c21fbca0cb8d6"
name="INSULATION RUN" type="normal"/>
  <objecttypeinfo id="UIDabf9e6-1152dc8e79a-f528764d624db129b32c21fbca0cb8d6"
name="BREAKOUT_TAPE" type="normal"/>
  <objecttypeinfo id="UIDabf9e6-1152dc8e79b-f528764d624db129b32c21fbca0cb8d6" name="SPOT_TAPE"
type="normal"/>
  <objecttypeinfo id="UIDabf9e6-1152dc8e79c-f528764d624db129b32c21fbca0cb8d6"
name="INTERCONNECTCONDUCTOR" type="normal"/>
  <objecttypeinfo id="UIDabf9e6-1152dc8e79d-f528764d624db129b32c21fbca0cb8d6" name="MLC"
type="normal"/>
  <objecttypeinfo id="UIDabf9e6-1152dc8e79e-f528764d624db129b32c21fbca0cb8d6"
name="OTHERCOMPONENT" type="normal"/>
  <objecttypeinfo id="UIDabf9e6-1152dc8e79f-f528764d624db129b32c21fbca0cb8d6" name="NODE"
type="normal"/>
</project>
```

If the requested project belongs to a domain which is not assigned to the Capital user used to authenticate the request, then this web service returns a SOAP fault.

```
SOAP Fault = Project with specified attributes cannot be found or access not granted to <user>
```

Using the Usage Definitions exporter service

Name	ExportDeviceUsages
Request schema	$CAPITAL_HOME/dtd/bridgesdeviceusage.xsd
Response schema	$CAPITAL_HOME/dtd/bridgesdeviceusage.xsd
Response	Capital-XML representation of Usage Definitions

See the provided ExportDeviceUsageClient example Java class for how to invoke this service and how to process the response.

The request payload needs to include the root element as per the XML schema, with identification of the project by name. The root element can be sent with no attribute in which case System Usage Definitions will be output.
The request can also filter for specific Library part numbers of which usages are to be retrieved by providing *usagedef* children element with their *name* attribute (which should match the Library part number). If no part filter is specified, then all usages are retrieved.

```
<project name="example1">
   <usagedef name="dev1"/>
</project>
```

The response payload contains all Usage definitions for the parts specified in the request filter (or all if no filter was specified).

```
<project xsi:schemaLocation="http://www.mentor.com/harness/Schema/bridgesdeviceusage
H:\capital_home/dtd/bridgesdeviceusage.xsd" author="system" creationDate="2007-11-12"
creationTime="14:17:14" schemaVersion="1 0" id="UIDabf9e6-1152dc8e716-
f528764d624db129b32c21fbca0cb8d6" name="example1"
xmlns="http://www.mentor.com/harness/Schema/bridgesdeviceusage">
  <usagedef id="UID8f606b-11633a82637-f07a5839dcbf9e609596bd40e7930890" modified="2" name="dev1"
libraryref="UIDcf427b-11633b4abfa-f07a5839dcbf9e609596bd40e7930890" baseid="UID8f606b-
11633a82637-f07a5839dcbf9e609596bd40e7930890">
        <deviceusage id="UID8f606b-11633a82638-f07a5839dcbf9e609596bd40e7930890" name="DEV1"
baseid="UID8f606b-11633a8263a-f07a5839dcbf9e609596bd40e7930890">
                <pinusage id="UID8f606b-11633a82639-f07a5839dcbf9e609596bd40e7930890" name="1"
baseid="UID8f606b-11633a82639-f07a5839dcbf9e609596bd40e7930890">
                        <pinusagenetname netname="net1"/>
                </pinusage>
                <pinusage id="UID8f606b-11633a8263a-f07a5839dcbf9e609596bd40e7930890" name="2"
baseid="UID8f606b-11633a8263a-f07a5839dcbf9e609596bd40e7930890">
                        <pinusagenetname netname="net2"/>
                </pinusage>
        </deviceusage>
  </usagedef>
</project>
```

If the requested project belongs to a domain which is not assigned to the Capital user used to authenticate the request, then this web service returns a SOAP fault.

```
SOAP Fault = Project with specified attributes cannot be found or access not granted to <user>
```

Using the Usage Definitions Import service

Name	ImportUsageDefinitions
Request schema	$CAPITAL_HOME/dtd/project.dtd
Response schema	None
Response	Capital-XML representation of Usage Definitions import feedback

This web service enables a user to import usage definitions at project or system level.

The request payload needs to include the root element as per the XML schema, with identification of the project by name or into system in the absence of project name. The root element can be sent with no “project” attribute in which case System Usage Definitions import is assumed.

Here are the details how request needs to be constructed and response needs to be parsed

Web service request

```
<import project=”projectName” replace=”true/false”>
</import>
```

If project name is blank then it’s assumed as system usages import

The actual data to import (XML symbol library) must be provided as an XML SOAP attachment.

The following is a sample import file containing project usages

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE usagemgr PUBLIC "-//Mentor Graphics Corporation//Project 1.109//EN">
<usagemgr id="_1" modified="4" baseid="_1">
  <usagedef id="_2" modified="2" name="SysDev01" libraryref="UID476610-12b9f287fe9-
121dedf945d7c55d48682816d928ea46" baseid="_2">
    <deviceusage id="_3" name="Sys01" baseid="_3">
      <pinusage id="_4" name="1" baseid="_4"/>
    </deviceusage>
  </usagedef>
  <usagedef id="_9" modified="2" name="SysDev02" libraryref="UID476610-12b9f287ff5-
121dedf945d7c55d48682816d928ea46" baseid="_9">
    <deviceusage id="_10" name="Sys01" baseid="_10">
      <pinusage id="_11" name="1" baseid="_11"/>
    </deviceusage>
  </usagedef>
</usagemgr>
```

Note the following from the above given example

- DOCTYPE contains usagemgr entry and
- All the usage definitions are embedded in usagemgr element

And the following is a sample import file containing system usages

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE systemusagemgr PUBLIC "-//Mentor Graphics Corporation//Project 1.102//EN" "U:/capital_home/dtd/project.dtd">
<systemusagemgr id="_1" modified="12" baseid="_1">
  <usagedef id="_2" modified="2" name="SysDev01" libraryref="UID476610-12b9f287fe9-121dedf945d7c55d48682816d928ea46" baseid="_2">
    <deviceusage id="_3" name="Sys01" baseid="_3">
      <pinusage id="_4" name="1" baseid="_4"/>
    </deviceusage>
  </usagedef>
  <usagedef id="_9" modified="2" name="SysDev02" libraryref="UID476610-12b9f287ff5-121dedf945d7c55d48682816d928ea46" baseid="_9">
    <deviceusage id="_10" name="Sys01" baseid="_10">
      <pinusage id="_11" name="1" baseid="_11"/>
    </deviceusage>
  </usagedef>
</systemusagemgr>
```

Note the following from the above given example

- DOCTYPE contains systemusagemgr entry and
- All the usage definitions are embedded in systemusagemgr element

Web service response

```
<importfeedback status="success/fail" error="error message of failure" />
```

Error message will be blank on successful import.

The following are the steps to be followed to import project/system usages

1. Construct root element as explained above
2. Create a SOAP attachment containing the usages (either project or system usages but not both) data
3. Send the request
4. Parse the response for “status” attribute and “error” attribute in case of failure.

Please note that project is not reflected immediately with the imported usages. Project needs to be re-opened to get the imported usages. Project application needs to be closed and re-opened to get the imported system usages.

Using the Security data exporter service

Name	ExportSecurityData
Request schema	$CAPITAL_HOME/dtd/Security.xsd
Response schema	$CAPITAL_HOME/dtd/Security.xsd
Response	Capital-XML representation of security data

See the provided ExportSecurityDataClient example Java class for how to invoke this service and how to process the response.

The request payload needs to include the root element as per the XML schema as shown below

```
<SecurityMgr currentuser="true/false"/>
```

The response payload contains all security information, including users, permissions, roles, user groups and domains. The functionality is documented as part of Capital User.

When currentuser attribute is specified and is “true”, only the user, with which this service is requested, data will be exported. Default value of this attribute is “false”.

Note: in order to re-import this data through the Capital User interactive function, a DTD declaration must be inserted just before the root element:

```
<!DOCTYPE SecurityMgr PUBLIC "-//Mentor Graphics Corporation//SecurityMgr 1.0//EN"
"[$CAPITAL_HOME]/dtd/security.dtd">
```

Where the [$CAPITAL_HOME] field above must be substituted with the path of your Capital installation directory.

Using the Design release status update service

Name	DesignReleaseStatus
Request schema	$CAPITAL_HOME/dtd/bridgesdesignstatus.xsd
Response schema	-
Response	-

See the provided DesignReleaseStatusUpdateClient example Java class for how to invoke this service.

The request payload needs to include the root element as per the XML schema, with identification of the design: sufficient attributes must be provided to uniquely identify it. For instance, if only the *name* (and *projectname*) is provided and it matches more than one design, then a SOAP fault is returned. If the *id* attribute is provided, then no other attribute needs to be provided as it uniquely identifies the design. Note also that the design is uniquely identified by the (*name*, *revision*, *short description*) triplet within a given project.
Despite the name of the root element (*wiringdesign*), all types of designs are supported (Logic, Integrator or Harness designs).
The root element *releasestatus* attribute must specify the target release status (i.e. Release Level) by name (not by type).
The optional root element *freezesharedobjectsused* may be specified to freeze the shared objects while the design release.

```
<wiringdesign name="Design1" projectname="example1" releasestatus="Intermediate”
freezesharedobjectsused="true”/>
```

The service carries out the following checks before actually changing the release status of the design:

- If the design project belongs to a domain which is not assigned to the Capital user used to authenticate the request, a SOAP fault is returned:

```
SOAP Fault = Design with specified attributes cannot be found in accessible domains
```

- The target status must exist and a transition from the current status to the target status must exist, otherwise a SOAP fault is returned:

```
SOAP Fault = There is no possible release level transition from <current level> to <target level>
for design <design name>
```

- The target status must be different than the current status, otherwise a SOAP fault is returned:

```
SOAP Fault = <design name> design release level is already <current level>
```

- The user used to authenticate the request must have the permission to change the status to the target status, otherwise a SOAP fault is returned:

```
SOAP Fault = User <user name> does not have permission to change design status to <status type>
```

- If the release level allows “Frozen Shared Objects” behavior and a transition from the current status to the target status with *freezesharedobjectsused*="false”, a SOAP fault is returned

```
SOAP Fault = Release design with unfrozen shared objects operation is not allowed for design
<design name> at release level <target level>
```

- If the project preference “Enforce Design Rule Checks on release of designs” is enabled and if the target release level is of type Released, an error is returned

```
SOAP Fault = Design not released as the project preference enforces Design Rule Checks to be run
on release of designs.
```

While changing the release status, any hook associated with this event will be triggered (see How to implement the design release status hook).

The response payload is null.

Using the Wiring properties update web service

Name	BridgeInWiring
Request schema	$CAPITAL_HOME/dtd/bridgesconnectivity.xsd
Response schema	$CAPITAL_HOME/dtd/bridgesreport.xsd
Response	Raw XML bridge report

See the provided WiringPropertiesUpdateClient example Java class for how to invoke this service.

The request payload needs to include the root element as per the XML schema, with identification of the design: sufficient attributes must be provided to uniquely identify it. For instance, if only the *name* (and *projectname*) is provided and it matches more than one design, then a SOAP fault is returned. If the *id* attribute is provided, then no other attribute needs to be provided as it uniquely identifies the design. Note also that the design is uniquely identified by the (*name*, *revision*, *short description*) triplet within a given project.
Only Logic designs and build lists are supported.

```
<wiringdesign name="Design1" projectname="example1"> <wire id="UIDe69c0a-1152dcc76d3-
f528764d624db129b32c21fbca0cb8d6" name="WIRE1" wirelength="0.75"/></wiringdesign>
```

This service applies the attributes and properties of the wiring elements found in the incoming request payload and applies them to the corresponding Capital wiring objects. Equivalent functionality can be found in the Capital Bridges, by bridging in wiring data using the Capital 3D Wiring format.
The example above simply changes the value of the Wire Length attribute from 0.0 to 0.75 for wire WIRE1, which can be found in Diagram1. The wire is matched by its id attribute and can be imported in any revision of the design in which it exists, then all other attributes which value differs are applied to the matched wire.

The service carries out the following checks before actually changing the wiring objects in Capital:

- If the design project belongs to a domain which is not assigned to the Capital user used to authenticate the request, a SOAP fault is returned:

```
SOAP Fault = Design with specified attributes cannot be found in accessible domains
```

- If the design release level does not include the Design Change behavior, a SOAP fault is returned:

```
SOAP Fault = Design change operation is not allowed for design <design name> at release level
<level name>
```

- If the design release level does not include the Bridge behavior, a SOAP fault is returned:

```
SOAP Fault = Bridge design operation is not allowed for design <design name> at release level
<level name>
```

- If the design is locked, a SOAP fault is returned:

```
SOAP Fault = <design name> is locked. Write operation cancelled.
```

The response payload contains the Bridge report in raw XML form. It can be transformed to its usual HTML form using the $CAPITAL_HOME/adaptors/resources/default_report.xslt XSLT transformation.

```
<report author="system" creationDate="2007-11-10" creationTime="21:48:23" schemaVersion="1_0"
xsi:schemaLocation="http://www.mentor.com/harness/Schema/bridgesreport
[$CAPITAL HOME]/dtd/bridgesreport.xsd"
xmlns="http://www.mentor.com/harness/Schema/bridgesreport">
        <header>
                <source>Remote connection: 127.0.0.1</source>
                <target>Design: Design1:1:Logic design example 1</target>
                <start_time>10 November 2007 21:48:24 GMT</start_time>
                <units>
                        <unit>Length unit: Meter (m)</unit>
                </units>
                <title>Wiring Bridge Web Service Report</title>
                <format>X2ML</format>
                <codetrans>
                        <codetrans>No</codetrans>
                </codetrans>
                <parttrans>
                        <parttrans>No</parttrans>
                </parttrans>
                <end_time>10 November 2007 21:48:34 GMT</end_time>
        </header>
        <log>
                <message type="0">
                        <description>Number of imported Wire: 1</description>
                        <remedial/>
                </message>
        </log>
        <summary>
                <error_count>Number of errors: 0</error_count>
                <warning_count>Number of warnings: 0</warning_count>
        </summary>
</report>
```

Using the Option information importer service

Name	ImportOptions
Request schema	$CAPITAL_HOME/dtd/bridgesoptions.xsd
Response schema	-
Response	-

See the provided ImportOptionsClient example Java class for how to invoke this service.

The request payload contains the options information to import as per the XML schema. The root element identifies the project into which this information is to be imported. If project identification is not provided, then the service will attempt to import the information in the System Data.

```
<project name='example1'>
<optionmgr id='_1'>
        <option id='_2' name='opt1' description='option 1 modified' isvariant='true'/>
</optionmgr>
</project>
```

This service executes the import of options information in the same way as Capital Project does, i.e. if an option with the same name already exists in the target project, then its attributes will be updated as per the incoming XML request payload.

If the requested project belongs to a domain which is not assigned to the Capital user used to authenticate the request, then this web service returns a SOAP fault.

```
SOAP Fault = Project with specified attributes cannot be found or access not granted to <user>
```

Note: in order to import data exported from the Capital Project interactive function, the following DTD declaration should be removed:

```
<!DOCTYPE optionmgr PUBLIC "-//Mentor Graphics Corporation//OptionMgr 1.0//EN"
"[$CAPITAL_HOME]/dtd/options.dtd">
```

Using the Object Type information importer service

Name	ImportObjectTypeInfo
Request schema	$CAPITAL_HOME/dtd/bridgesobjecttypeinfo.xsd
Response schema	-
Response	-

See the provided ImportObjectTypeInfoClient example Java class for how to invoke this service.

The request payload contains the Object Type information to import as per the XML schema. The root element identifies the project into which this information is to be imported. If project identification is not provided, then the service will attempt to import the information in the System Data.

```
<project name="example1">
<objecttypeinfo id="_1" name="DEVICE" type="normal">
    <nametemplate id="_2" name="DEV1" description="device 1 modified" shortdescription="dev 1
modif"/>
</objecttypeinfo>
</project>
```

This service executes the import of options information in the same way as Capital Project does, i.e. if a definition with the same name already exists in the target project, then its attributes will be updated as per the incoming XML request payload.

If the requested project belongs to a domain which is not assigned to the Capital user used to authenticate the request, then this web service returns a SOAP fault.

```
SOAP Fault = Project with specified attributes cannot be found or access not granted to <user>
```

Using the Library importer service

Name	ImportLibraryParts
Request schema	$CAPITAL_HOME/dtd/LibrarySchema.xsd
Response schema	Not provided
Response	Short import report

See the provided ImportLibraryPartsClient example Java class for how to invoke this service.

The request schema only describes the content to be imported. It must be wrapped into a higher level element which specifies the import parameters. Those are the same parameters as for importing from the Capital Library interactive application. Attribute overwritemode can have two values truncate or merge.

To import XML data:

```
<importservice>
<import content="xml" overwritecodedesc="false" overwritepart="false" scopeimport="false"
overwritemode="truncate">
        [ importing XML content to be inserted here ]
</import>
</importservice>
```

To import text data:

```
<importservice>
<import content="txt" overwritecodedesc="false" overwritepart="false" scopeimport="false"
overwritemode="truncate">
        [ importing TXT content to be inserted here ]
</import>
</importservice>
```

The example client program retrieves the XML data (resulting from a previous export from Capital Library) to be imported from the sample files provided, inserts it in the <importservice> header element as described above and sends this as the request to the web service.

The web service returns a response which indicates the status of the import.

```
<importservice xmlns="http://www.mentor.com/harness/Schema/ImportServiceSchema"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" SchemaValidation="true">
        <importfeedback status="success"/>
</importservice>
```

Using the Design revisionning service

Name	DesignRevision
Request schema	$CAPITAL_HOME/dtd/bridgesdesignstatus.xsd
Response schema	-

Response	-

See the provided DesignRevisionningClient example Java class for how to invoke this service.

The request payload needs to include the root element as per the XML schema, with identification of the design: sufficient attributes must be provided to uniquely identify it. For instance, if only the *name* (and *projectname*) is provided and it matches more than one design, then a SOAP fault is returned. If the *id* attribute is provided, then no other attribute needs to be provided as it uniquely identifies the design. Note also that the design is uniquely identified by the (*name*, *revision*, *short description*) triplet within a given project.
Despite the name of the root element (*wiringdesign*), all types of designs are supported (Logic, Integrator or Harness designs).

```
<wiringdesign name="Design1" projectname="RevisionExample" revision="1" revisionname="2"
revisionpartnumber="" revisionshortdesc="short desc" shortdesc=""/>
```

The service carries out the following checks before actually revising a design:

- If the user doesn't have the permission to revise a design, a SOAP fault is returned:

```
SOAP Fault = User <user name> does not have permission to revise a design
```

- If the release level of the parent design doesn't allow creating a revision, a SOAP fault is returned:

```
SOAP Fault = Revision not allowed for this design release level
```

- The triplet (name, revision, short description) of the new revision must be unique in the project, otherwise a SOAP fault is returned:

```
SOAP Fault = The triplet(name,revision,short description) of the new revision must be unique in
the project
```

- The user should access only the designs within his domain, otherwise a SOAP fault is returned:

```
SOAP Fault = Design not found in database accessible domains
```

The response payload is null.

Using the Project Template exporter service

Name	ExportProjectTemplate
Request schema	$CAPITAL_HOME/dtd/bridgesdesignstatus.xsd

Response schema	$CAPITAL_HOME/dtd/project.dtd
Response	Capital-XML representation of the project template

See the provided ExportProjectTemplateClient example Java class for how to invoke this service and how to process the response.

The request payload needs to include the root element as per the XML schema, with identification of the project, by *name*. Note that only the root element for this schema is used in this request – and is actually the same as other equivalent schemas bridgesobjecttypeinfo.xsd and bridgesoptions.xsd.

```
<project name="example1"/>
```

The response payload is based on the Capital project DTD from which the <!DOCTYPE> declaration has been removed for SOAP compatibility. Therefore, an extra attribute *dtdversion* is added to the root *project* element to indicate the DTD version. This allows to correctly upgrade the XML when re-importing the design in a higher level of Capital.

```
<project baseid="_1" description="" domainref="" dtdversion="1.56" id="_1" modified="13"
name="TestProj">
…
```

Note: in order to re-import this data through the Capital Project interactive function, a DTD declaration must be inserted just before the root element:

```
<!DOCTYPE project PUBLIC "-//Mentor Graphics Corporation//Project [version]//EN"
"[$CAPITAL_HOME]/dtd/project.dtd">
```

Where the [$CAPITAL_HOME] field above must be substituted with the path of your Capital installation directory and the [version] field by the value of the *dtdversion* attribute.

Using the System Data exporter service

Name	ExportSystemData
Request schema	None
Response schema	$CAPITAL_HOME/dtd/project.dtd
Response	Capital-XML representation of the system data

See the provided ExportSystemDataClient example Java class for how to invoke this service and how to process the response.

The request payload needs to include the root element as per the XML schema, with identification of the systemdata. Note that only the root element for this schema is used in this request.

```
<systemdata/>
```

The response payload is based on the Capital project DTD from which the <!DOCTYPE> declaration has been removed for SOAP compatibility. Therefore, an extra attribute *dtdversion* is added to the root *system_data* element to indicate the DTD version. This allows to correctly upgrade the XML when re-importing the system data in a higher level of Capital.

```
<system_data dtdversion="1.56" id="UIDdb38a4-ff35b1af20-f528764d624db129b32c21fbca0cb8d6">
```

Note: in order to re-import this data through the Capital Project interactive function, a DTD declaration must be inserted just before the root element:

```
<!DOCTYPE system_data PUBLIC "-//Mentor Graphics Corporation//Project [version]//EN"
"[$CAPITAL_HOME]/dtd/project.dtd">
```

Where the [$CAPITAL_HOME] field above must be substituted with the path of your Capital installation directory and the [version] field by the value of the *dtdversion* attribute.

Using the Library Parts exporter service

Name	ExportLibraryParts
Request schema	$CAPITAL_HOME/dtd/libraryexportparts.xsd
Response schema	$CAPITAL_HOME/dtd/LibrarySchema.xsd
Response	Capital-XML representation of the library data

See the provided ExportLibraryPartsClient example Java class for how to invoke this service and how to process the response.

The request payload includes all the library parts which need to be exported as per the XML schema with identification of part by *partnumber* or *uid.*
Please note that if both partnumber and uid is mentioned, then the query will be done based on the partnumber.

```
<Parts includescoping="true" exportrevisions="true" exportallcodes="false">
<Part partnumber="connector01">
<Part uid=" UIDdb38a4-ff35b1af20-f528764d624db129b32c21fbca0cb8d6">
</Parts>
```

The response will contain all the details for the library parts which are found in the library. If no library part is found, only the header information will be there in the response. The response will adhere to the XML schema mentioned above.This service can also be used to extract the full set of library codes using the “exportallcodes” attribute with the value “true”. Should this attribute be true then the service will export all library codes along with parts.

Using the Design Bridge Out service

Name	BridgeOutDesign
Request schema	$CAPITAL_HOME/dtd/bridgesdesignstatus.xsd
Response schema	$CAPITAL_HOME/dtd/bridgesconnectivity.xsd $CAPITAL_HOME/dtd/bridgesharness.xsd
Response	Bridged out design XML (Capital 3D Wiring or Capital 3D Harness)

See the provided BridgeOutDesignClient example Java class for how to invoke this service and how to process the response.

The request payload needs to include the root element as per the XML schema, with identification of the design: sufficient attributes must be provided to uniquely identify it. For instance, if only the *name* (and *projectname*) is provided and it matches more than one design, then a SOAP fault is returned. If the *id* attribute is provided, then no other attribute needs to be provided as it uniquely identifies the design. Note also that the design is uniquely identified by the (*name*, *revision*, *short description*) triplet within a given project.
Despite the name of the root element (*wiringdesign*), all types of designs are supported (Logic, Integrator or Harness designs). Additionally, Logic build lists are also supported.

```
<wiringdesign designtype="" name="Design1" projectname="test1" revision="" shortdesc=""/>
```

For a Logic build list, use *designtype* buildlist.

```
<wiringdesign designtype="buildlist" name="List1" projectname="test1" revision="" shortdesc=""/>
```

The response payload contains the bridged out XML data, as per the Capital 3D Wiring bridge format if the design is a Logic design (or build list), or as per the Capital 3D Harness bridge format id the design is a Harness design.

Using the Design importer service

Name	ImportDesign
Request schema	$CAPITAL_HOME/dtd/bridgesimportdataset.xsd $CAPITAL_HOME/dtd/project.dtd for attachments
Response schema	
Response	

See the provided ImportDesignClient example Java class for how to invoke this service and how to process the response.

The request payload needs to include the root element as per the XML schema, with identification of the target project, using the *designprojectname* attribute. The following options can be specified as attributes to the root element (if unspecified, the default value is *true*):

- *designcopyasrevision* (*true* / *false*): The designs are imported as revisions of existing designs (value is ignored if *designoverwriterevision* is true)
- *designcopyassociated* (*true* / *false*): For an Integrator design, if associated Logical designs are provided, those will be imported as well
- *designcopygeneratedwiring* (*true* / *false*): For an Integrator design, if generated wiring is included, it will be imported
- *designoverwriterevision* (*true* / *false*): If a design revision already exists in the target project, then it will be overwritten
- *designcopyeffectivity (true / false): If set to true effectivity values of design will be copied.*

```
<import designprojectname="example1" />
```

The request message must also include a single gzip attachment which contains the design XML data – only one design import is allowed per request.

The response is empty unless the service raises a SOAP fault, in which case the whole transaction is aborted.

Using the Symbol exporter service

Name	SymbolLibraryExport
Request schema	$CAPITAL_HOME/dtd/bridgessymbollibrary.xsd
Response schema	$CAPITAL_HOME/dtd/symbollib.dtd
Response	

See the provided ExportSymbolClient example Java class for how to invoke this service and how to process the response.

This service returns an empty response (unless a SOAP fault is raised): the payload is in fact included in an XML attachment.

The request payload needs to include the root element as per the XML schema, with identification of the library to import (by either *name* or *uid*). Optionally, *symbol* children element can be added (also specified by either *name* or *uid*) for a selective export. If no children elements are provided, then the whole library will be exported.

- *compressed (true / false, default:false): When this is true, the symbol library is sent to the client in compressed (gzip) format. This is useful to avoid network traffic to send considerably big XML content*

```
<symbollibrary name="symbollib" compressed='false'/>
```

The response payload is included in an XML attachment which binds to the standard Capital symbollib DTD and can be imported as such in the interactive Capital Symbol or Capital Project applications.

Note that the user used to authenticate the request must have the permission to export symbols, otherwise a SOAP fault is returned.

Using the Symbol importer service

Name	SymbolLibraryImport
Request schema	$CAPITAL_HOME/dtd/bridgesimportdataset.xsd $CAPITAL_HOME/dtd/symbollib.dtd for attachments
Response schema	
Response	

See the provided ImportSymbolClient example Java class for how to invoke this service and how to process the response.

The request payload needs to include the root element as per the XML schema, the following options can be specified as attributes to the root element (if unspecified, the default value is *true*):

- *symboloverwrite* (*true* / *false*): If the symbol already exists in the target database, then it is overwritten with the incoming data.
- *compressed (true / false, default:false): When this is true, the symbol library XML is sent to the server in compressed (gzip) format. This is useful to avoid network traffic to send considerably big XML content*

```
<import symboloverwrite=”false” compressed=’false’ />
```

Additionally, the actual data to import (XML symbol library) must be provided as an XML SOAP attachment.

Note that the user used to authenticate the request must have the permission to import symbols, otherwise a SOAP fault is returned.

Using the Language Dictionary Export service

Name	ExportLanguageDictionary
Request schema	
Response schema	
Response	Language Dictionary in csv format

See the provided ExportLanguageDictionaryClient example Java class for how to invoke this service and how to process the response.

```
<languagemgr/>
```

This service returns an empty response (unless a SOAP fault is raised): the payload is included in a zipped attachment which includes a language dictionary in csv format which can be imported in the interactive Capital Project applications.

Using the Language Dictionary importer service

Name	ImportLanguageDictionary
Request schema	
Response schema	
Response	

See the provided ImportLanguageDictionaryClient example Java class for how to invoke this service and how to process the response.

The request payload needs to include the zipped language dictionary (csv format) as a SOAP attachment.

```
<dictionary/>
```

Note that the user used to authenticate the request must have the permission to import language dictionary, otherwise a SOAP fault is returned.

Using the EngineeringChangeOrders exporter service

Name	ExportEngineeringChangeOrders
Request schema	$CAPITAL_HOME/dtd/ bridgesengineeringchangeorderstatus.xsd
Response schema	$CAPITAL_HOME/dtd/bridgesengineeringchangeorders.xsd
Response	Capital-XML representation of EngineeringChangeOrder

See the provided ExportEngineeringChangeOrdersClient example Java class for how to invoke this service and how to process the response.

The request payload needs to include the root element as per the XML schema, with identification of the project by name.
The following options are provided as attributes to the engineeringchangeorder element

- asattachment – default value is “false”. If this parameter is specified with true value, then the response XML will be exported as gzip attachment.
- category - If this attribute is specified, then all the Engineering Change Orders which belong to this category will be exported. In the absence of this attribute, the category will not be considered for filtering the Engineering change orders
- releaselevelname – If this attribute is specified then all the Engineering change orders having the specified release level will be filtered and exported. In the absence of this attribute, release level will not be considered for filtering the Engineering change orders

NOTE: If both the attributes (category & releaselevelname) are specified, then all the Engineering change orders belonging to that category and having the specified release level will be exported.

CAUTION: If both the attributes (category & releaselevelname) are not specified, it will be an unconstrained query and will take more time and memory based on the number of Engineering change orders in the system.

```
<project name="example1">
<engineeringchangeorder asattachment="false” category="TestCategory2" releaselevelname="Draft"/>
</project>
```

The response payload contains details regarding engineering change orders

```
<?xml version="1.0" encoding="UTF-8"?><project
xmlns="http://www.mentor.com/harness/Schema/engineeringchangeorder"
dtdversion="EngineeringChangeOrderMgr 1.0" name="ECOWebServiceTestProject">
  <engineeringchangeordermgr baseid="UID9b6da0-130e0fbbc9c-6a8e08dc23d7014bc90dcd07d29e216c"
id="UID9b6da0-130e0fbbc9c-6a8e08dc23d7014bc90dcd07d29e216c" modified="12">
    <engineeringchangeorder author="superuser" baseid="UID9b6da0-130e0fbbcbf-
6a8e08dc23d7014bc90dcd07d29e216c" category="TestCategory2" created="1309444850557"
description="description 3" id="UID9b6da0-130e0fbbcbf-6a8e08dc23d7014bc90dcd07d29e216c"
modificationtimestamp="1309445000615" modificationuser="superuser" modified="4" name="Test-ECO3"
releaselevelname="Draft" shortdescription="short description 3">
      <property name="Prop2" stability="editable" type="String" val="testVal2"/>
      <engineeringchangeorderassociation associatedid="UID9b6da0-130e0fbbca1-
6a8e08dc23d7014bc90dcd07d29e216c" associationtype="Logical" completionstatus="notstarted"/>
    </engineeringchangeorder>
    <engineeringchangeorder author="superuser" baseid="UID9b6da0-130e0fbbcc1-
6a8e08dc23d7014bc90dcd07d29e216c" category="TestCategory2" created="1309444856598"
description="description 4" id="UID9b6da0-130e0fbbcc1-6a8e08dc23d7014bc90dcd07d29e216c"
modificationtimestamp="1309445000596" modificationuser="superuser" modified="4" name="Test-ECO4"
releaselevelname="Draft" shortdescription="short description 4">
      <engineeringchangeorderassociation associatedid="UID9b6da0-130e0fbbca1-
6a8e08dc23d7014bc90dcd07d29e216c" associationtype="Logical" completionstatus="notstarted"/>
      <engineeringchangeorderassociation associatedid="UID9b6da0-130e0fbbcb3-
6a8e08dc23d7014bc90dcd07d29e216c" associationtype="Harness" completionstatus="notstarted"/>
    </engineeringchangeorder>
  </engineeringchangeordermgr>
</project>
```

If the requested project belongs to a domain which is not assigned to the Capital user used to authenticate the request, then this web service returns a SOAP fault.

```
SOAP Fault = Project with specified attributes cannot be found or access not granted to <user>
```

Using the EngineeringChangeOrders importer service

Name	ImportEngineeringChangeOrders
Request schema	$CAPITAL_HOME/dtd/engineeringchangeorder.dtd
Response schema	-
Response	-

See the provided ImportEngineeringChangeOrdersClient example Java class for how to invoke this service.

The request payload contains the Engineering change orders to import as per the XML schema. The root element identifies the project into which this information is to be imported.

```
<project name='example1'>
<engineeringchangeordermgr baseid="UID9b6da0-130e0fbbc9c-6a8e08dc23d7014bc90dcd07d29e216c"
id="UID9b6da0-130e0fbbc9c-6a8e08dc23d7014bc90dcd07d29e216c" modified="12">
    <engineeringchangeorder author="superuser" baseid="UID9b6da0-130e0fbbcbb-
6a8e08dc23d7014bc90dcd07d29e216c" category="TestCategory1" created="1309444830372"
description="description 1" id="UID9b6da0-130e0fbbcbb-6a8e08dc23d7014bc90dcd07d29e216c"
modificationtimestamp="1309445000633" modificationuser="superuser" modified="4" name="TEST-ECO1"
releaselevelname="Draft" shortdescription="imported successfully">
      <property name="P1" stability="editable" type="Integer" val="11"/>
      <property name="Prop2" stability="editable" type="String" val="testVal2"/>
      <engineeringchangeorderassociation associatedid="UID9b6da0-130e0fbbcb3-
6a8e08dc23d7014bc90dcd07d29e216c" associationtype="Harness" completionstatus="notstarted"/>
    </engineeringchangeorder>
  </engineeringchangeordermgr>
</project>
```

If an Engineering change order with the same name already exists in the target project, then its attributes will be over written with the incoming XML request payload.

NOTE: The user needs to have the Engineering change order creation and Engineering change order modification permissions. In the absence of these permissions, this web service returns a SOAP fault with appropriate message.

If the requested project belongs to a domain which is not assigned to the Capital user used to authenticate the request, then this web service returns a SOAP fault.

```
SOAP Fault = Project with specified attributes cannot be found or access not granted to <user>
```

Using the Submit Task service

Name	SubmitTask
Request schema	$CAPITAL_HOME/dtd/bridgessubmittask.xsd
Response schema	-

Response	-

See the provided SubmitTaskClient example Java class for how to invoke this service and how to process the response.

The request payload needs to include the root element as per the XML schema, the following options are specified as attributes to the root element (if unspecified, the default value is *true*):

- *cron_expression*: The cron expression is used to describe the schedule for running the task. If you don't specify anything then task will be executed as soon as possible. If you require assistance in the construction of a cron expression, then there are several good resources available on the internet, eg *www.cronmaker.com*.
  - Example cron expression to run a task at specified time say 4.20 PM, 24th October 2012 is "00 20 16 24 10 ? 2012"
  - Example cron expression to run task 10 PM every Wednesday and Saturday is "0 0 22 ? * WED,SAT *"
  - Example cron expression to run task 12 PM every day is "0 0 12 1/1 * ? *"
- *instance_name*: The name that is to be given to this submitted instance of the task.
- *name*: This is the name of the Capital task that is to be submitted, eg HarnessProcessingTask, SBOMProcessingTask. For the custom task the plugin URL for the task should be used
- *Id*: This is an optional attribute and if provided an existing task with this UID will be modified.
- *application* : This is an optional attribute and can be used to specify the application using which task needs to be run. Value to be provided in this field is the real application name as seen in capital_home\bin folder (example CapitalModularXC.exe)

```
<taskspec cron_expression="0 0/2 * 1/1 * ? *" instance_name="ProcessDoorHarnes"
name="HarnessProcessingTask"/>
```

The task parameters must be provided as an XML SOAP attachment. Content ID and type for attachment should be given as below. Note that ID should be FEMParam and this is used to differentiate parameter attachment from other attachments which can be supplied

- Content-Type: application/xml
- Content-ID: FEMParams

The following is a sample set of task parameters for a harness processing task:

```
<?xml version="1.0" encoding="UTF-8"?>
<harnessprocessingsettings>
  <project projectname="Quick Start - Automotive Interactive"/>
  <compositedesign designname="DOOR-DR" revision="A"    />
  <moduleparentdesign designname="DOOR-DR" revision="A"/>
  <enggservice isengineeringautomaticallypositionsplices="true" isengineeringbalancesplices="true"
isengineeringcalculatebundlesizes="true" isengineeringcalculatecenterstripdistances="true"
```

```
isengineeringcalculateharnessweight="true" isengineeringcalculateinsulationtotals="true"
isengineeringcalculatewirelengths="true" isengineeringselectcavitycomponents="true"
isengineeringselectcustomerpartnumbers="true" isengineeringselectheatshrinkpartnumbers="true"
isengineeringselectmandatorypartnumbers="true" isengineeringselectmulticorepartnumbers="true"
isengineeringselectsplicepartnumbers="true" isengineeringselectspottapepartnumbers="true"
isengineeringselectsupplierpartnumbers="true" isengineeringselecttubepartnumbers="true"
isengineeringselectwirepartnumbers="true" isforcecommondirectionforcenterstripsplices="true"
isrecalculateallsplicewireroutes="true"/>
        <harnesschecksservice isbranchesswithnowiresormulticores="true" iscavitieswithblankterminalmaterial="true"
iscavitieswithplugsorsealsmarkedasnone="true" iscenterstriporidcmidwirenotonwireroute="true"
iscomponentfitspositionrule="true" iscomponenthousingdefinitionvalidity="true" iscomponentnotonanybranch="true"
isconnectorswithdifferentterminalmaterials="true" isgeneratereport="false" iscavitycomponentsnotfittingwires="true"
iscavityplugsandwiresatsamecavity="true" isinvalidorunselectedpartnumbers="true" isoptionexpressionvalidity ="true"
isunsourcedpartnumbers ="true" isunterminatedwireends ="true" isincludeblanksections ="false"
ismandatoryitemselectionvalidity="true" ismultiplecomponentsatpositionrule="true" issealedandunsealedconnectors="true"
isvalidwirepath="true" iswireorinsulationnoderoutevalidity="true" iswiresfitcavities ="true" iswireswithzerostriplength ="true"/>
      </harnessprocessingsettings>
```

There is a provision to provide additional task attachments. Only certain task (for example, a custom task) will be reading this and appropriate SOAP attachment as needed by the task can also be provided. Content ID for these attachments should be as needed by the task so that it is identified properly. Content ID for these attachments shouldn't be "FEMParam" as it is reserved for parameter xml and it shouldn't start with "capital" as it is reserved for capital use.

Using the List Tasks service

Name	ListTasks
Request schema	$CAPITAL_HOME/dtd/bridgeslisttasksrequest.xsd
Response schema	$CAPITAL_HOME/dtd/bridgeslisttasks.xsd
Response	XML with list of tasks

See the provided ListTaskClient example Java class for how to invoke this service and how to process the response.

The request payload needs to include the root element as per the XML schema.

The criteria can be used to filter the results based on the

- Name: This is the name of the Capital task that is to be submitted, e.g. HarnessProcessingTask, SBOMProcessingTask. For the custom task the plugin URL for the task should be used.
- Status: (e.g. PENDING, COMPLETED)

```
<criteria><names> <name value='HarnessProcessingTask'/></names><statuses> <status
value='PENDING'/> </statuses></criteria>
```

Name and status will be AND'ed. Multiple names and statuses can be provided. Multiple Names\Statuses will be ORed between themselves

Using the Describe Task service

Name	DescribeTask
Request schema	$CAPITAL_HOME/dtd/bridgesdescribetaskrequest.xsd
Response schema	$CAPITAL_HOME/dtd/bridgesdescribetask.xsd
Response	XML providing the task details

See the provided DescribeTaskClient example Java class for how to invoke this service and how to process the response.

The request payload needs to include the root element as per the XML schema, the following options can be specified as attributes to the root element:

- *id*: The task must be identified in the request by its UID. This may have been previously retrieved by the ListTasks web service.
- *attachments* (*true* / *false*): If the task was submitted with attachments then these can be retrieved by the request. Note that these will come as SOAP attachment.
    - Client program using this service should retrieve the attachment they are interested in using Content-ID of the attachment. Note that there can be more attachments in future for the same task and hence the client program should gracefully handle and pick the ones it needs
    - Content-ID starting with “capital” is reserved for capital. Currently there is one such special type “capital_htmloutput” and it contains html output for the last task execution in zipped format
- *executions* (*true* / *false*): Specifies whether the scheduled task execution descriptions should be retrieved.
- *parameters* (*true* / *false*): Specifies whether the parameter which describe the task should be retrieved. It will be retrieved as SOAP attachment.

```
<taskspec attachments='false' executions='true' id='UIDb560c2-132720ad3d5-
f441900c27154aedfdf3a803036ea5f8' parameters='true'/>
```

Using the Delete Task service

Name	DeleteTask
Request schema	$CAPITAL_HOME/dtd/bridgesdeletetask.xsd
Response schema	-
Response	-

See the provided DeleteTaskClient example Java class for how to invoke this service and how to process the response.

The request payload needs to include the root element as per the XML schema. The task must be identified in the request by its UID.

```
<taskspec id='UIDb560c2-132720ad469-f441900c27154aedfdf3a803036ea5f8' forceDelete="true"/>
```

The forceDelete attribute is optional. Specifying this attribute with value true will permanently delete the task along with its all execution records. If this attribute is not specified or value provided is false then task is moved to STOPPED state to avoid future executions.
If the task hasn't run yet then the task is always deleted irrespective of the forceDelete attribute value.

Note that the user used to authenticate the request must have either the task administration permission or he should be the one who had submitted the task, otherwise a SOAP fault is returned.

Using the Release Multiple Designs service

Name	ReleaseMultipleDesigns
Request schema	$CAPITAL_HOME/dtd/ bridgesreleasemultipledesignsrequest.xsd
Response schema	$CAPITAL_HOME/dtd/ bridgesreleasemultipledesignsresponse.xsd
Response	XML providing the list of failed designs

See the provided ReleaseMultipleDesignsClient example Java class for how to invoke this service.

The request payload needs to include the root element as per the XML schema, with identification of the project. The optional root element *freezesharedobjectsused* may be specified to freeze the shared objects while the design release.

The child elements contain the design information where sufficient attributes must be provided to uniquely identify it. For instance, if only the *name* is provided and it matches more than one design, then that design will not be processed. If the *id* attribute is provided, then no other attribute needs to be provided as it uniquely identifies the design. Note also that the design is uniquely identified by the (*name*, *revision*, *short description*) triplet within a given project.
Despite the name of the child element (*wiringdesign*), all types of designs are supported (Logic, Integrator or Harness designs).
The child elements *releasestatus* attribute must specify the target release status (i.e. Release Level) by name (not by type).

```
<project name="TestProject">
    <wiringdesign id="UID1…." releasestatus="Released" />
```

```
    <wiringdesign id="UID2…." releasestatus="Released″ />
    <wiringdesign id="UID3…." releasestatus="Released″ />
    <wiringdesign id="UID4…." releasestatus="Released″ />
</project>
```

All the designs will be processed sequentially and all the failures will be reported as a response to the request.

The service carries out the following checks before actually changing the release status of each individual design:

- If the design project belongs to a domain which is not assigned to the Capital user used to authenticate the request, an error code is returned:

```
SOAP Fault = Design with specified attributes cannot be found in accessible domains
```

- The target status must exist and a transition from the current status to the target status must exist, otherwise an error is returned for that design:

```
Error = There is no possible release level transition from <current level> to <target level> for
design <design name>
```

- The target status must be different than the current status, otherwise a an error is returned for that design:

```
Error = <design name> design release level is already <current level>
```

- The user used to authenticate the request must have the permission to change the status to the target status, otherwise an error is returned for that design:

```
Error = User <user name> does not have permission to change design status to <status type>
```

- If the release level allows “Frozen Shared Objects” behavior and a transition from the current status to the target status with *freezesharedobjectsused*="false”, an error is returned

```
Error = Release design with unfrozen shared objects operation is not allowed for design <design
name> at release level <target level>
```

- If the project preference “Enforce Design Rule Checks on release of designs” is enabled and if the target release level is of type Released, an error is returned

```
Error = Design not released as the project preference enforces Design Rule Checks to be run on
release of designs.
```

While changing the release status, any hook associated with this event will be triggered (see How to implement the design release status hook).

The response payload is as follows:

```
<project name="TestProject">
        <wiringdesign id="UID1…." releasestatus="Released" error=" There is no possible release
level transition from <current level> to <target level> for design <design name>" />
</project>
```

Using the Build List Release Level Change service

Name	BuildListReleaseLevelChange
Request schema	$CAPITAL_HOME/dtd/bridgesbuildlistreleaselevelrequest.xsd
Response schema	$CAPITAL_HOME/dtd/bridgesbuildlistreleaselevelrequest.xsd
Response	XML providing the list of failed Build Lists

See the provided BuildListReleaseStatusChangeClient example Java class for how to invoke this service.

The request payload needs to include the root element as per the XML schema, with identification of the project.

The child elements contain the Build List information where sufficient attributes must be provided to uniquely identify it. For instance, if only the *name* is provided and it matches more than one Build List types, then that Build List will not be processed. If the *id* attribute is provided, then no other attribute needs to be provided as it uniquely identifies the Build List. Note also that the Build List is uniquely identified by the (*name* and *type*) pair within a given project.

The child elements *releasestatus* attribute must specify the target release status (i.e. Release Level) by name (not by type).

```
<project name="TestProject">
    <buildlist id="UID1…." releasestatus="Released" />
    <buildlist id="UID2…." releasestatus="Released" />
    <buildlist id="UID3…." releasestatus="Pending" />
    <buildlist id="UID4…." releasestatus="Intermediate" />
</project>
```

All the Build Lists will be processed sequentially and all the failures will be reported as a response to the request.

The service carries out the following checks before actually changing the release status of each individual design:

- The target status must exist and a transition from the current status to the target status must exist, otherwise an error is returned for that design:

```
Error = There is no possible release level transition from <current level> to <target level>
```

- The target status must be different than the current status, otherwise a an error is returned for that Build List:

```
Error = <design name> design release level is already <current level>
```

- The user used to authenticate the request must have the permission to change the status to the target status, otherwise an error is returned for that Build List:

```
Error = User <user name> does not have permission to change Build List status to <status type>
```

While changing the release status, any hook associated with this event will be triggered (see How to implement the design release status hook).

The response payload is as follows:

```
<project name="TestProject">
        <buildlist id="UID1…." releasestatus="Released" error=" There is no possible release level
transition from <current level> to <target level>" />
</project>
```

DRC as Task

It is possible to run DRCs on any design, buildlist or SBOM by submitting task. You have to give appropriate application name to run and list of DRCs as part the task parameters.

The name of task is DRCTask and it takes a config file which has details of target and list of DRCs that should be run on the target. Please see the example file DRCProcessingTask.xml in {capital.home}\doc\webservice\data, which has list of all available DRCs.

The following is a sample config file for a DRC task:

```
<?xml version="1.0" encoding="UTF-8"?>
<drcsettings project="exampleproject" drcobject="UID6a6ebd-13e5c321c48-
87fecf32183b52214ad60e22ba7de368" result="$CHS_HOME/temp/drc_task_result.csv">
  <drctorun drcname="Logic.Checks.DanglingNetCheck" severity="Error" title="Net not terminating
at a pin"/>
  <drctorun drcname="Logic.Checks.DanglingShieldCheck" severity="Error" title="Shield termination
not terminating at a pin"/>
  <drctorun drcname="Logic.Checks.plugin:Java//com.mentor.drc.ExampleCustomDRC"
severity="Warning" title="Custom DRC"/>
/drcsettings>
```

Below is the explanation of each attribute in config xml file

Project: The name of the project on which the DRC task needs to run

DRCObject: This is uid of the design, buildlist or SBOM on which DRCs needs to run. UID can be obtained by looking at project xml file. You can also find design uid using listdesigns webservice

Result: This is the location where result of DRC run can be written.
DRCName: This is the unique identifier for a DRC. You can look at DRCProcessingTask.xml to find identifier for all available DRCs in capital

Severity: Severity of each DRC. The valid values for this are Error, Warning, Information

The following is a sample result csv file:

```
"Error","Invalid name: P801 is not on the list of valid Connector names","Naming","Invalid name
for component","7-Power Mirrors_Wiring","P801"

"Warning","Device DOOR ECU has no symbol","Inclusion","Missing symbol for device","7-Power
Mirrors_Wiring","DOOR ECU"
```

Release Designs as Task

It is possible to release any design by submitting the *ReleaseDesignsTask*. It takes an appropriate application name and a configuration file which has details of design revisions to be transitioned to target release level. Please see the example file ReleaseDesignsTask.xml in {capital.home}\doc\webservice\data, which has a sample format to release multiple designs.

```
<?xml version="1.0" encoding="UTF-8"?>
<project projectname="example1" annotate="false" warningaserror="false">
  <wiringdesign name="Design1" revision="1" releasestatus="Complete"/>
  <wiringdesign name="Design2" revision="1" releasestatus="Complete"/>
</project>
```

Below is the explanation of each attribute in configuration xml file

projectname: Name of the project under consideration.

annotate: Defines if the diagrams need to be annotated after the release of the design. Accepts true/false.

warningaserror: Defines if the warnings that arise out of design rule checks to be treated as errors. Accepts true/false.

name: Name of the design to be released.

revision: Revision of the design to be released.

shortdesc: Short description of the design to be released.

partnumber: Part number of the harness design to be released.

releasestatus Target release level of type Released to which the design needs to be transitioned.

The following is a sample result:

```
Mon Nov 18 15:38:50 IST 2013 -- Started execution of the task ReleaseDesignsTask
Task - Release Designs
Task Execution ID - UID3c9e58-1426aaebc2f-2d74541398def4af87da30952f02f321
Task Parameter File - E:\chs_home\temp\ReleaseDesignsTask_1426aaebc2f.xml
User - system
PROGRESS - Start Phase Releasing Design: Design1:1 ---- (1)
OUTPUT - Releasing Design: Design1:1
OUTPUT - Release Designs summary
   Design   Status   Comments
   Design1:1   Succeeded   Design transitioned to "Complete" release level.
OUTPUT - Writing DRC results to "E:\chs_home\temp\UID3c9e58-1426aaebc2f-
2d74541398def4af87da30952f02f321.csv"
PROGRESS - Phase - 1 Step ---- (1/1)
Mon Nov 18 15:38:53 IST 2013 -- Finished execution of the task ReleaseDesignsTask
```

Using the BuildLists list service

Name	ListBuildLists
Request schema	$CAPITAL_HOME/dtd/bridgesListBuildListsRequest.xsd
Response schema	$CAPITAL_HOME/dtd/bridgesListBuildListsResponse.xsd
Response	List of Build Lists meta data

See the provided ListBuildListsClient example Java class for how to invoke this service and how to process the response.

```
<?xml version="1.0" encoding="UTF-8"?>
<project name='testProject'>
        <buildlisttype>logic</buildlisttype>
        <buildlisttype>harness</buildlisttype>
</project>
```

The response payload provides the list of Build Lists as children of the root element.

Following is a sample response showing all the metadata information of each BuildList.

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<buildlists xmlns="http://www.mentor.com/harness/Schema/bridgesListBuildListsResponse"
author="system" creationDate="2014-07-31" creationTime="18:03:05" schemaVersion="1_0">
  <buildlist buildlistmaintenance="MANUAL" description="" effectivityvalue="" folderpath=""
id="UID4e2806-1476cdbac10-2d74541398def4af87da30952f02f321" modified="6" name="logicBuildList"
releaselevel="Draft" type="logic"/>
```

```
  <buildlist buildlistmaintenance="MANUAL" description="logic build list 2" effectivityvalue=""
folderpath="f1" id="UIDaee210-1478bfa9601-2d74541398def4af87da30952f02f321" modified="3"
name="logicBuildList2" releaselevel="Draft" type="logic"/>
</buildlists>
```

If the requested project belongs to a domain which is not assigned to the Capital user used to authenticate the request, then this web service returns a SOAP fault.

```
SOAP Fault = Project with specified attributes cannot be found or access not granted to <user>
```

Using the Build List Importer service

Name	ImportBuildLists
Request schema	$CAPITAL_HOME/dtd/bridgesBuildListsExportRequest.xsd
Response schema	$CAPITAL_HOME/dtd/bridgesimportbuildlistsresponse.xsd
Response	XML providing the list of failed Build Lists

See the provided ImportBuildListsClient example Java class for how to invoke this service and how to process the response.

The following options are provided as attributes to the project element

- overwritebuildlists – default value is “false”. By default import of existing Build Lists will be skipped. If this parameter is specified with true value, then the existing Build Lists in the project will be overwritten provided the existing Build List release level allows editing.

The import of Build Lists is atomic in nature. If one of the BuildLists fail to import, none of the BuildLists in the request will be imported.

The request payload contains the Build Lists to import as per the XML schema. The root element identifies the project into which this information is to be imported.

Following is a sample request to create a Build List

```
<?xml version="1.0" encoding="UTF-8"?>
<project name="Sample">
  <buildlistmgr id="_1">
    <buildlist buildlistmaintenance="MANUAL" description="" effectivityvalue=""
folderpath="Sample\Sample1" id="_2" name="HarnBuildList" releaselevel="Draft" type="harness">
      <buildlistdetail design="UIDc4ee53-147431320b3-d24e0123c53c2f7d4a85ff466f43f062" id="_2"/>
      <ecoref buildlist_id="_2" id="UID4a045c-133b6e9569f-0ccf12edafedf30717990c6b7fe0c1bc"/>
      <property name="prop1" stability="editable" type="String" val="propvalue"/>
    </buildlist>
  </buildlistmgr>
</project>
```

Following is a sample request to import the Build List in overwrite mode.

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<project xmlns="http://www.mentor.com/harness/Schema/bridgesbuildList" name="example4"
overwritebuildlists="true">
  <buildlistmgr id="_1">
    <buildlist buildlistmaintenance="MANUAL" description="" effectivityvalue="" folderpath=""
id="_2" name="logicBuildList" releaselevel="Draft" type="logic">
```

```
      <buildlistdetail design="UID4a045c-133b6e956a4-0ccf12edafedf30717990c6b7fe0c1bc" id="_2"/>
      <ecoref buildlist_id="_2" id="UID4a045c-133b6e9569a-0ccf12edafedf30717990c6b7fe0c1bc"/>
    </buildlist>
  </buildlistmgr>
</project>
```

Following is a sample response payload in case of a successful import of all the Build Lists in the request.

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<project xmlns="http://www.mentor.com/harness/Schema/bridgesimportbuildlistsresponse"
author="system" creationDate="2014-07-31" creationTime="16:13:02" dtdversion="BuildListMgr 1.0"
name="example4" schemaVersion="1_0"/>
```

Following is a sample response in case of a failure during import.

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<project xmlns="http://www.mentor.com/harness/Schema/bridgesimportbuildlistsresponse"
author="system" creationDate="2014-07-31" creationTime="16:13:02" dtdversion="BuildListMgr 1.0"
name="example4" overwritebuildlists="true" schemaVersion="1_0">
    <buildlist error="Build List logicBuildList : release level  Draft2 is not available."
name="logicBuildList" type="logic"/>
</project>
```

If the requested project belongs to a domain which is not assigned to the Capital user used to authenticate the request, then this web service returns a SOAP fault.

```
SOAP Fault = Project with specified attributes cannot be found or access not granted to <user>
```

Using the Build List exporter service

Name	ExportBuildListService
Request schema	$CAPITAL_HOME/dtd/bridgesBuildListsExportRequest.xsd
Response schema	$CAPITAL_HOME/dtd/bridgesbuildList.xsd
Response	Capital-XML representation of BuildList

See the provided ExportBuildListClient example Java class for how to invoke this service and how to process the response.

The request payload needs to include the root element as per the XML schema, with identification of the project by name.
The following options are provided as attributes to the Build List element

- asattachment – default value is “true”. The BuildList is exported as gzip attachment by default. If this parameter is specified with false value, then the response XML will be exported as xml file.
- name – Name of the BuildList that needs to be exported.
- type – Type of the BuildList that needs to be exported. E.g., logic/harness/analysis/topology.
- id – ID of the BuildList that needs to be exported.

Sample request:

```
<project name="example4">
<buildlist asattachment="false" name="logicBuildList" type="logic"/>
</project>
```

Following is a sample response payload that contains details of the exported BuildList.

```
<?xml version="1.0" encoding="UTF-8"?><project
xmlns="http://www.mentor.com/harness/Schema/bridgesbuildList" author="system" creationDate="2014-
07-31" creationTime="16:24:03" dtdversion="BuildListMgr 1.0" name="example4" schemaVersion="1_0">
  <buildlistmgr haschildren="true" id="UID4a045c-133b6e9567b-0ccf12edafedf30717990c6b7fe0c1bc"
modified="7">
    <buildlist buildlistmaintenance="MANUAL" description="" effectivityvalue="" folderpath=""
id="UID4e2806-1476cdbac10-2d74541398def4af87da30952f02f321" modified="4" name="logicBuildList"
releaselevel="Draft" type="logic">
      <buildlistdetail design="UID4a045c-133b6e956a4-0ccf12edafedf30717990c6b7fe0c1bc"
id="UID4e2806-1476cdbac10-2d74541398def4af87da30952f02f321"/>
      <ecoref buildlist_id="UID4e2806-1476cdbac10-2d74541398def4af87da30952f02f321"
id="UID4a045c-133b6e9569a-0ccf12edafedf30717990c6b7fe0c1bc"/>
      <ecoref buildlist_id="UID4e2806-1476cdbac10-2d74541398def4af87da30952f02f321"
id="UID4a045c-133b6e9569f-0ccf12edafedf30717990c6b7fe0c1bc"/>
    </buildlist>
  </buildlistmgr>
</project>
```

If the requested project belongs to a domain which is not assigned to the Capital user used to authenticate the request, then this web service returns a SOAP fault.

```
SOAP Fault = Project with specified attributes cannot be found or access not granted to <user>
```

Using the Batch Bridge In Task service

Name	BatchBridgeInTask
Request schema	$CAPITAL_HOME/dtd/bridgessubmittask.xsd
Response schema	-
Response	-

See the provided BatchBridgeInTaskClient example Java class for how to invoke this service and how to process the response.

A BatchBridgeInTask may be submitted using the *Submit Task* service.

The request payload needs to include the root element as per the XML schema, the following options are specified as attributes to the root element (if unspecified, the default value is *true*):

- *cron_expression*: The cron expression is used to describe the schedule for running the task. If you don't specify anything then task will be executed as soon as possible. If you require assistance in the construction of a cron

expression, then there are several good resources available on the internet, eg *www.cronmaker.com*.

  - Example cron expression to run a task at specified time say 4.20 PM, 24th October 2012 is “00 20 16 24 10 ? 2012”
  - Example cron expression to run task 10 PM every Wednesday and Saturday is “0 0 22 ? * WED,SAT *”
  - Example cron expression to run task 12 PM every day is “0 0 12 1/1 * ? *”

- *instance_name*: The name that is to be given to this submitted instance of the task.
- *name*: This is the name of the Capital task i.e. BatchBridgeInTask.
- *Id*: This is an optional attribute and if provided an existing task with this UID will be modified.
- *application* : This is an optional attribute and can be used to specify the application using which task needs to be run. Default value of this attribute is CapitalHarnessXC.exe. The other applicable value for this task is CapitalModularXC.exe.

```
<taskspec cron_expression="" instance_name="BridgeFEMTask" name="BatchBridgeInTask"/>
```

Input for BatchBridgeInTask is provided as 3 different XML SOAP attachments. Content-ID and Content-type for each of them are as per the following table:

S.No.	SOAP attachment description	Content-Type	Content-ID
1	Task parameters depicting key details like Project, Design, Adaptor etc.	application/xml	FEMParams
2	Bridge configuration file depicting various Bridge settings for Bridge In	application/xml	BRIDGE_CONFIG_FILE
3	Source adaptor file to be Bridged In	application/xml	BRIDGE_IMPORT_FILE

A sample FEMParams file that could be used is available here:
$CAPITAL_HOME\doc\webservice\data\BatchBridgeInTask_paramWithAttachments.xml

A sample BRIDGE_CONFIG_FILE is available here:
$CAPITAL_HOME\doc\webservice\data\BridgesTestConfigurations.xml

It is recommended that the required BRIDGE_CONFIG_FILE be automatically generated (by dumping from the existing Bridge Settings in the design tool). This can be done by building the sample Extensibility plugin “ExportBridgeConfigurationsAction” and running it from Capital Harness XC or Capital ModularXC. The sample plugin code is available here:
$CAPITAL_HOME\doc\plugin\examples\Java\src\com\example\plugin\bridge

Once the BatchBridgeInTask is complete, the Bridge adaptor report and the Change Manager / Bridge In report are saved as task attachments.

S.No.	BatchBridgeInTask artifact	Task attachment name
1	Bridge adaptor html report	CMAdaptorReport
2	Bridge In / Change Manager csv report	ChangeManagerReport

The *List Tasks* service may be used to list all the BatchBridgeInTask instances of a given state. The *Describe Task* service may be used to get the logs and attachments of any BatchBridgeInTask.

External Web Services deployed in CIS

It is possible to provide your own web services to deploy within CIS.
CIS is based on the Jetty technology which allows deploying any web service in the same container, by means of configuration.
Those additional web services are referred to as “external” web services as they are not provided as part of Capital.

How to configure CIS to expose external web services

As a web application container, CIS will include web services located in a specific directory.

This directory is prompted to the user as part of the Capital installation (specific CIS part of the installation), then will be persisted to the $CAPITAL_HOME/config/clientprops.xml file. The directory can subsequently be modified at any convenient time by manually editing the file as follows:

```
<webserver host="cis" port="49901" cisjettyroot="../adaptors/webapps"
embedded="false"/>
```

How to deploy Axis web services

Axis is a web service container application which allows client and server to interact directly with Java, abstracting away the SOAP layer.

The first step is to deploy the Axis web application in CIS:

- Download the latest version of Axis
- Copy the downloaded webapps/axis directory under $CAPITAL_HOME/adaptors/webapps
- Start CIS
- From your web browser, visit *http://localhost:49901/axis*
- If the Axis home page is displayed, it indicates a successful deployment of Axis into CIS: visit the “validate” link for further verification.

You can then deploy a Java web service simply by providing the Java file in the axis directory:

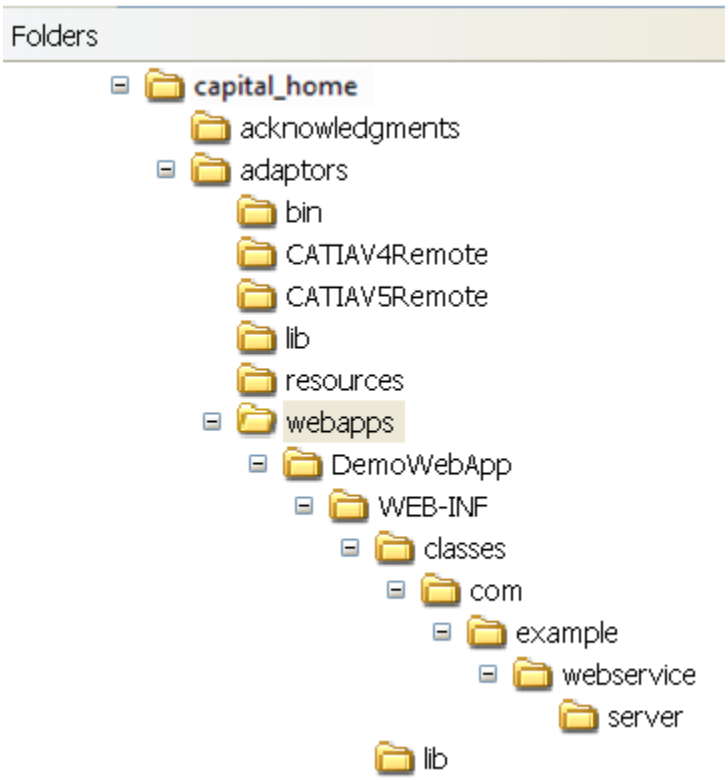
```
public class Calculator
{
      public int add(int i1, int i2)
      {
            Return i1 + i2;
      }

      public int substract(int i1, int i2)
      {
            Return i1 - i2;
      }
}
```

- Copy the above Java file (Calculator.jws) into $CAPITAL_HOME/adaptors/webapps/axis
- Start CIS
- From your web browser, visit *http://localhost:49901/axis/Calculator.jws*
- Axis automatically locates the file, compiles it and converts SOAP calls correctly into Java invocations of your service class.
- An example client class for this service can also be found in your Axis installation in the samples/userguide/example2/CalcClient.java file.

You can also find more information about Axis web services at:
http://www.developer.com/open/article.php/10930_3443951_1

How to deploy other web services

In the general case, you need to deploy a web application in CIS, which will act as your web services container.
If you have declared $CAPITAL_HOME/adaptors/webapps as your CIS deployment directory in $CAPITAL_HOME/config/clientprops.xml (see How to configure CIS to expose external web services), then you should build up the following directory structure:



- The *webapps* directory contains one sub-directory per deployed web application of the same name. There is only one web application *DemoWebApp* in the example provided.
- The web application directory (*DemoWebApp*) contains a *WEB-INF* sub-directory.
- The *WEB-INF* directory of a web application contains all the information about its servlets, i.e.:
    - The *classes* sub-directory which contains the Java classes that implement the servlets and web services
    - The *lib* sub-directory which contains the additional jar files used by the servlets

- The *web.xml* file which contains the web application and servlets deployment information in a standard format (http://java.sun.com/dtd/web-app_2_3.dtd)

How to build the example web service container application

You need the ant tool installed to build the Java samples with the build.xml file provided (otherwise, you can use your usual Java build process).

- From your shell command line, go to the $CAPITAL_HOME/doc/webservice/examples/server directory.
- Run “ant”: this will use the provided build.xml file to compile the Java classes, and create the *webapps* directory structure mentioned above in the $CAPITAL_HOME/adaptors directory. This task will also initially invoke the client examples build (as client code is also re-used in the web application example) and copy all sample and Capital required jar files to the *lib* directory.

How to run the example external web service

The example web service (DesignWireListReporterService) is invoked through a provided client sample, which can be run as any other CIS client (see How to run an example Java client):

- run.sh DesignWireListReporterClient

Architecture of the example external web service

Servlet

This is the Java class which implements the application various functions – in this case the only function is to invoke the requested web service and send a response back.
The ExampleServlet provided class extends the standard javax.servlet.HttpServlet class and implements the HTTP request-response methods (doGet(), doPost()): it extracts the SOAP document and payload from the incoming request, extracts the web service name from the URL used by the client, loads and invokes the corresponding web service class. It then wraps the web service response payload in a SOAP document and sends it back as an HTTP response.

Deployment descriptor file

The web.xml file contains the information necessary for CIS to find, load and execute the web application servlet(s).
The provided web.xml file describes the web application (name and description). It also associates a Java class and a URL pattern to the servlets.
The URL pattern describe which requests will be directed to each specific servlet, depending on the URL used by the client:
The “/*” pattern used in the example means that all URLs starting with: “http://<host>:<port>/DemoWebApp/” will be directed at the ExampleServlet servlet. This servlet then extract the remainder following this string and tries to load a web service class named after it.

Web Service

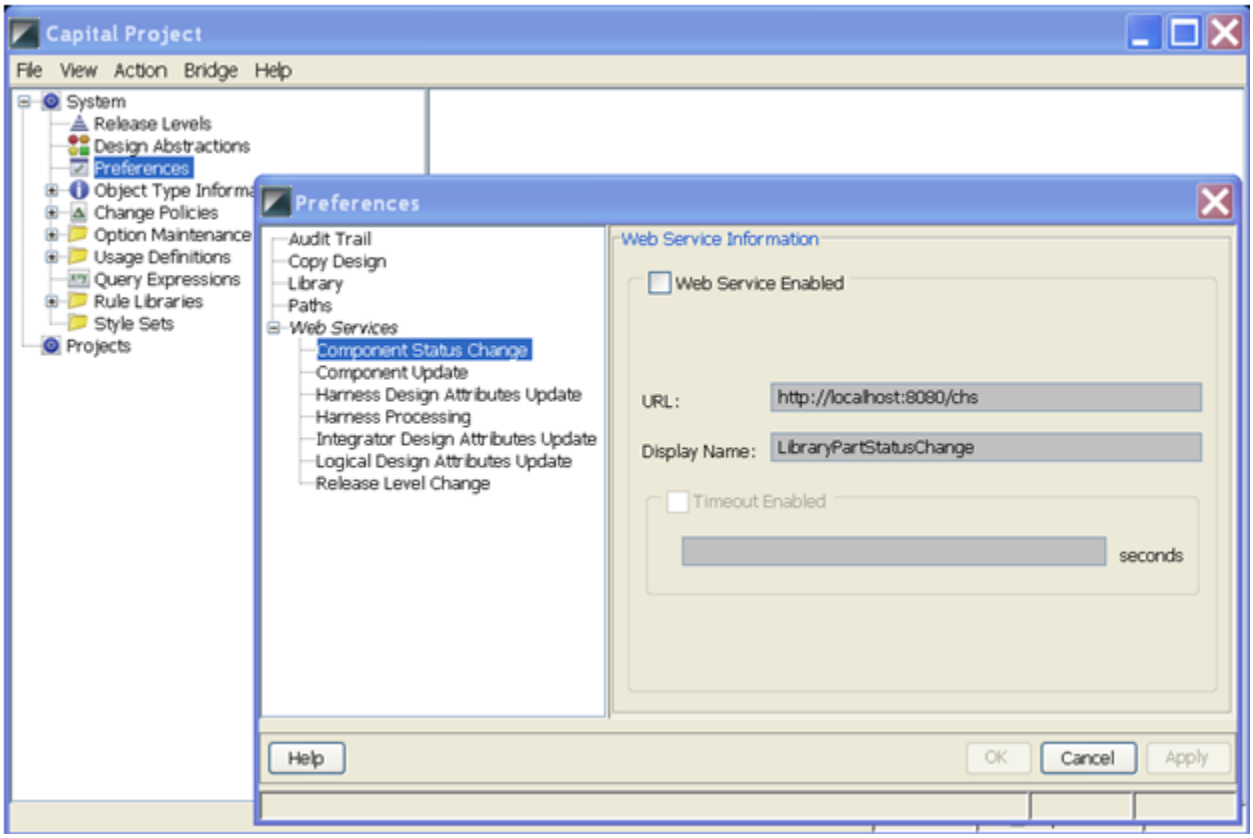
The example web service provided uses the same request schema as other CIS design based services (e.g. $CAPITAL_HOME/dtd/bridgesdesignstatus.xsd). It in turn invokes a CIS web service (design exporter service) by using the provided example client (ExportDesignClient), to extract the design data. It then parses this data to extract information about wires and their end point connections and stores it in a CSV file in a pre-defined directory. It does not send back a response.

Capital web service hooks

Capital provides a number of hooks – associated with certain Capital operations such as changing the release status of a design - which can be configured to invoke an external web service. This provides a means of extending Capital behavior or integrating with an external application as some of these hooks provide a bi-directional data transfer between Capital and the external application: data can be sent to the external web service (as part of the hook's request), modified by the external web service, sent back (as part of the response) and imported into Capital.

How to configure a hook

Each hook is listed in Capital Project under the System Preferences Web Services tree node.



Each hook presents the same main configuration information:

- Check the *Web Service Enabled* box to activate the hook. Activating the hook means an attempt will be made to invoke the external web service each time the Capital operation is carried out. The operation will fail if the attempt fails (e.g. the external web server that exposes the web service is not running) or if the web service returns a SOAP fault.
- Specify the *URL* of the external web service to be invoked. Note that the default value provided must always be modified as it does not correspond to any existing web service – but is merely a URL syntactic guide.
- Associate a *Display Name* with the hook: this name will be displayed only if the web service invocation can be explicitly triggered by some action button ("web service button") in the Capital user interface, in which case the display name will be shown on the button.

Additionally, the hooks which have an associated web service button use validation settings which control the triggering of the hook by both this button and the OK button of the form. The purpose is that data entered in the form cannot be saved (from the OK button) if it has not been validated by the invocation of the external web service – i.e. the invocation failed or the service returned a fault.

- *Independent* mode: both OK and web service buttons trigger the hook independently. The OK button additionally saves the data and closes the form.
- *Explicit* mode: The OK button is disabled until the web service button has been used to trigger the hook.
- *Implicit* mode: The web service button is not present. The OK button triggers the hook. The data is not saved and the form stays open if there was a problem invoking the service or it returned a fault.

How to implement the web service associated with a hook

How to implement a hook web service

The included Java examples provide all documentation necessary to implement any Capital hook web service.

These web services are deployed in CIS in the provided example, but could be deployed in any other web server. The example to implement, build and deploy those services has already been covered in How to deploy other web services. This should be done and CIS should be started prior to invoking any hook.

The data received by the Capital hook web service (request payload) is described in an XML schema published in $CAPITAL_HOME/dtd, as well as the data which it sends back (response payload) if there is a response.

Additionally, the type of transaction implemented by the client (see Web services protocol) is provided for each hook in the following paragraphs.

How to implement the design release status hook

Name	Release Level Change
Request schema	$CAPITAL_HOME/dtd/bridgesdesignstatus.xsd
Response schema	-
Response	-
Transaction type	Asynchronous fire-and-forget

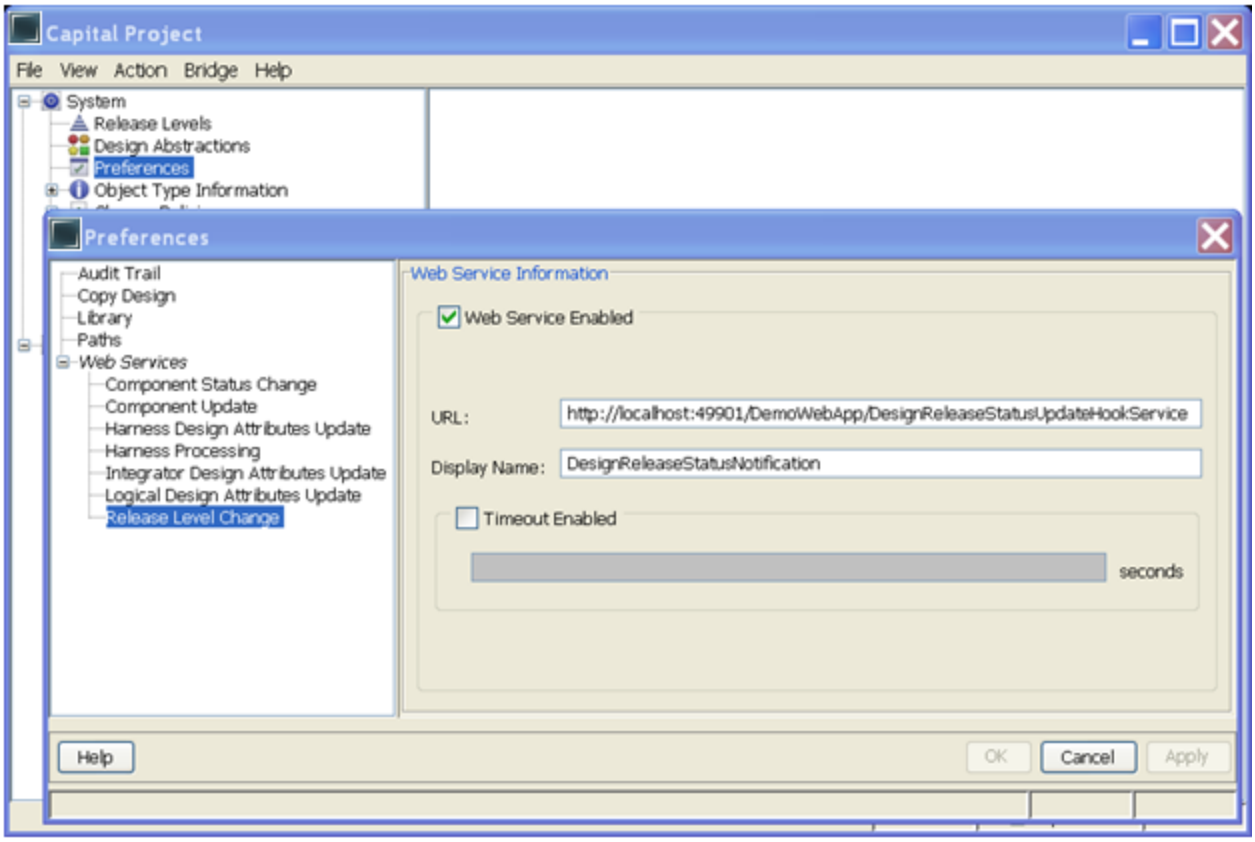
See the provided DesignReleaseStatusUpdateHookService example Java class for how to implement this service.

The request payload includes the design identification, the target release level name and the current (pre-transition) release level name. No response is expected.

The example implements a round-trip (see Integration patterns) by invoking a CIS web service to retrieve the SVG representation of any diagram in the design. It uses the existing client (see Using the SVG diagram generator service) and passes it the compatible incoming request which identifies the design.

To invoke this hook service:

- Configure the hook as follows:



- Edit the design and change the release status: the transition from *Intermediate* to *Complete* will result in the SVG diagram being stored in the C:\temp\CIS directory.

Once activated, the hook will be triggered wherever the release status of the design is changed, including in Capital Logic or in the CIS design release status web service.

How to implement the design attributes update hook

Name	Harness/Integrator/Logic Design Attributes Update
Request schema	$CAPITAL_HOME/dtd/bridgesdesignattributes.xsd
Response schema	$CAPITAL_HOME/dtd/bridgesdesignattributes.xsd
Response	Modified design attributes and properties
Transaction type	Synchronous request-response

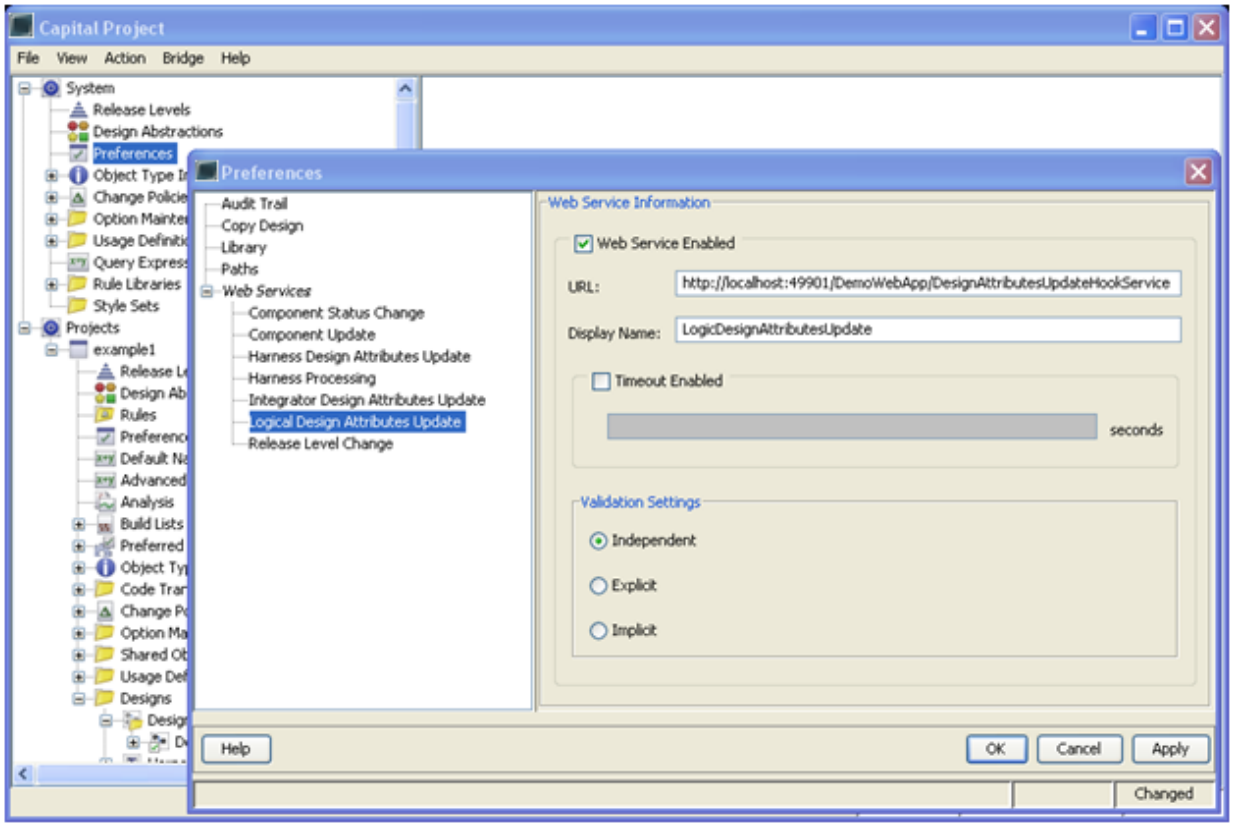
See the provided DesignAttributesUpdateHookService example Java class for how to implement this service.

The request payload includes the design identification, all its attributes and properties.

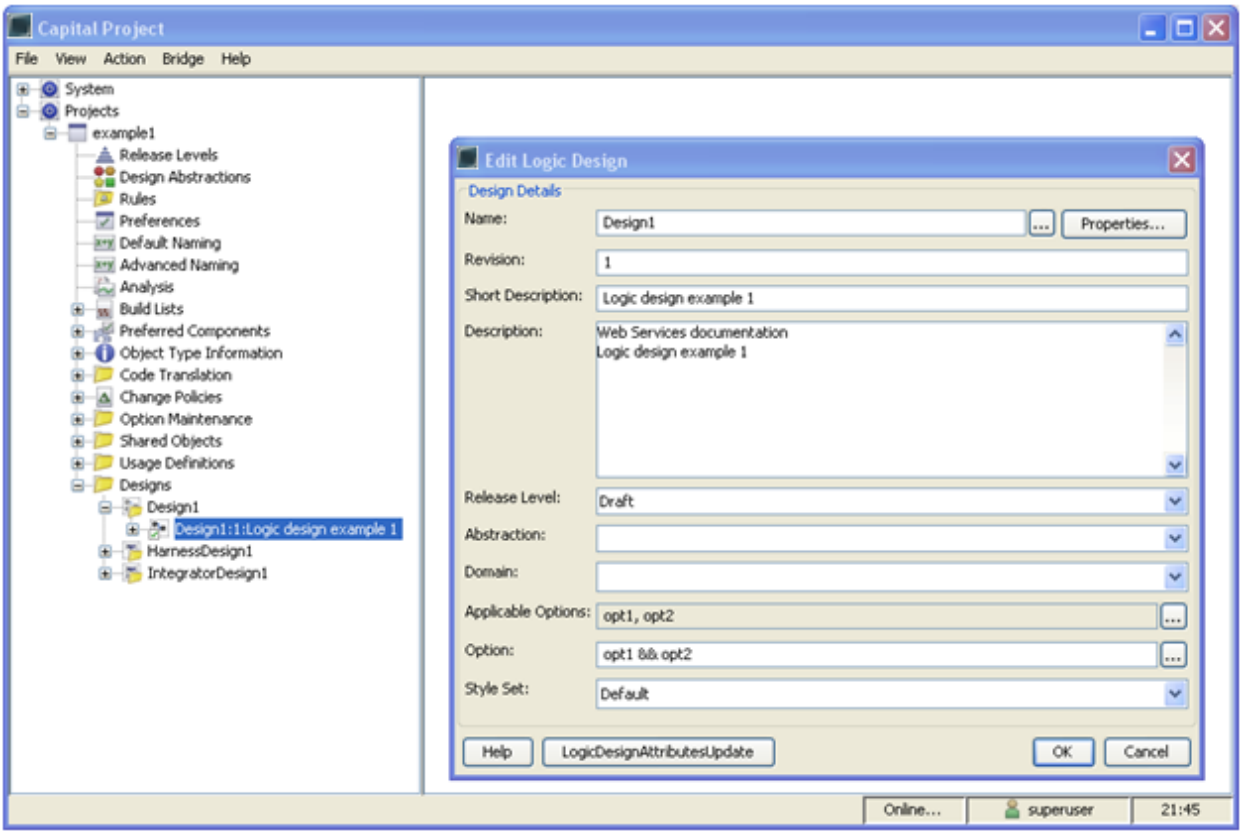
The example web service provided adds or updates a design property which name is “webservice validation” and value is the date and time at which it was set.

To invoke this hook service:

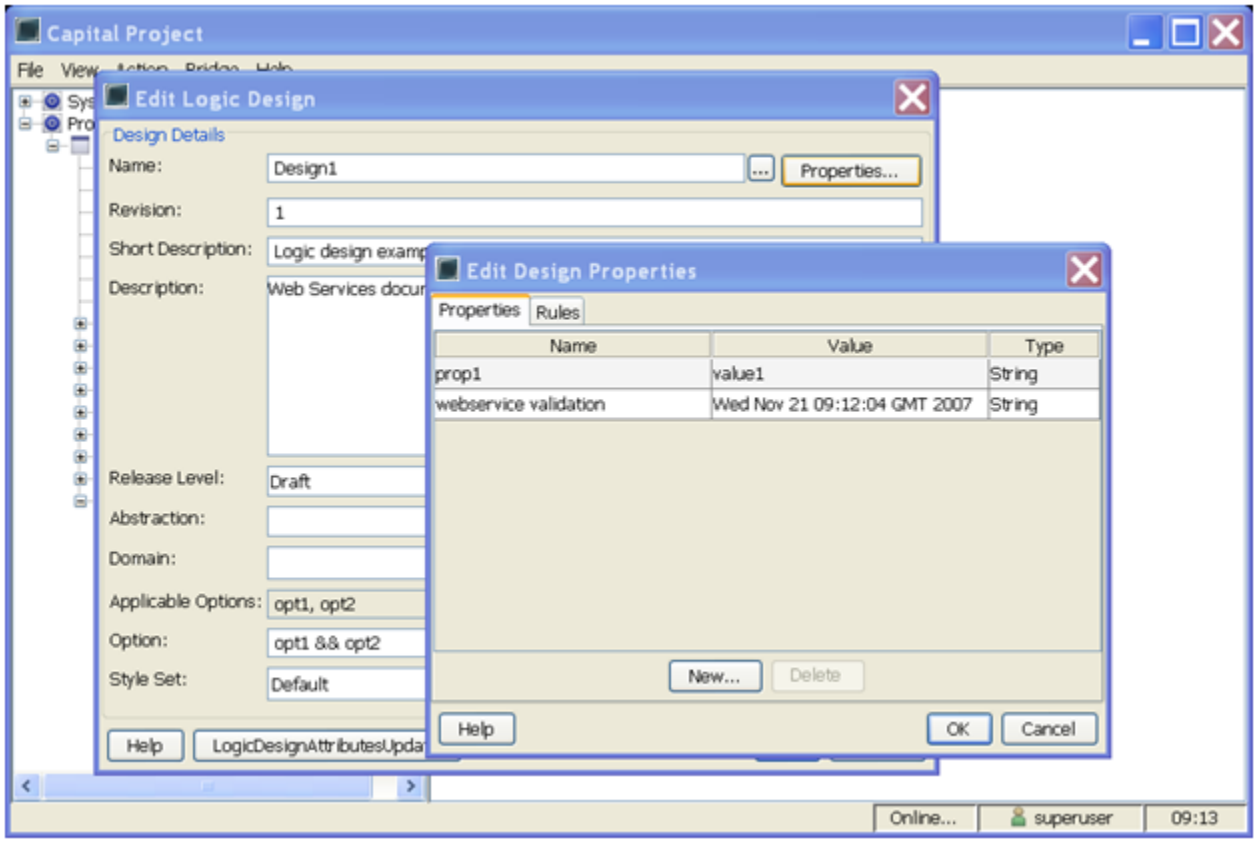
- Configure the hook as follows:



- Edit the design and click the “LogicDesignAttributeUpdate” button.



- To see the result, click the “Properties…” button



How to implement the library component update hook

Name	Component Update
Request schema	$CAPITAL_HOME/dtd/library.xsd
Response schema	$CAPITAL_HOME/dtd/library.xsd
Response	Modified component data
Transaction type	Synchronous request-response

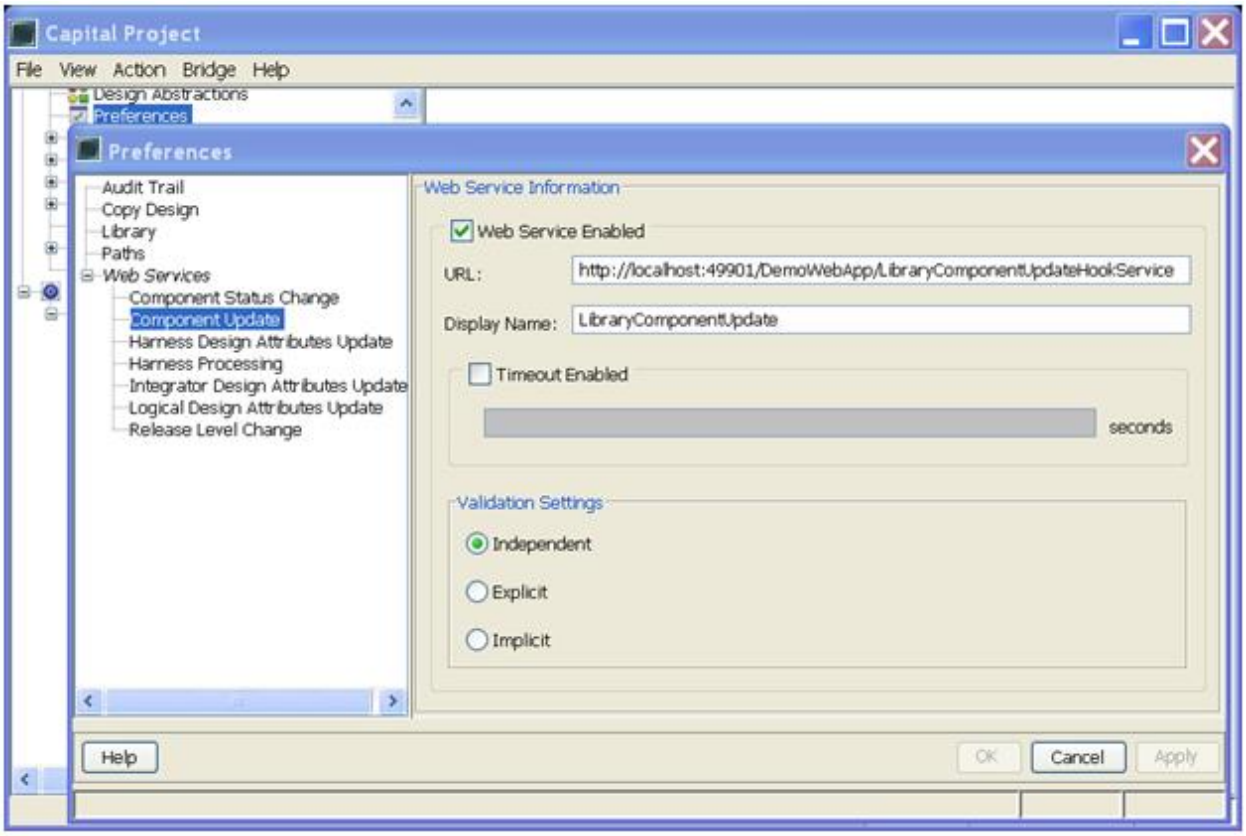
See the provided LibraryComponentUpdateHookService example Java class for how to implement this service.

The request payload includes the full component information.

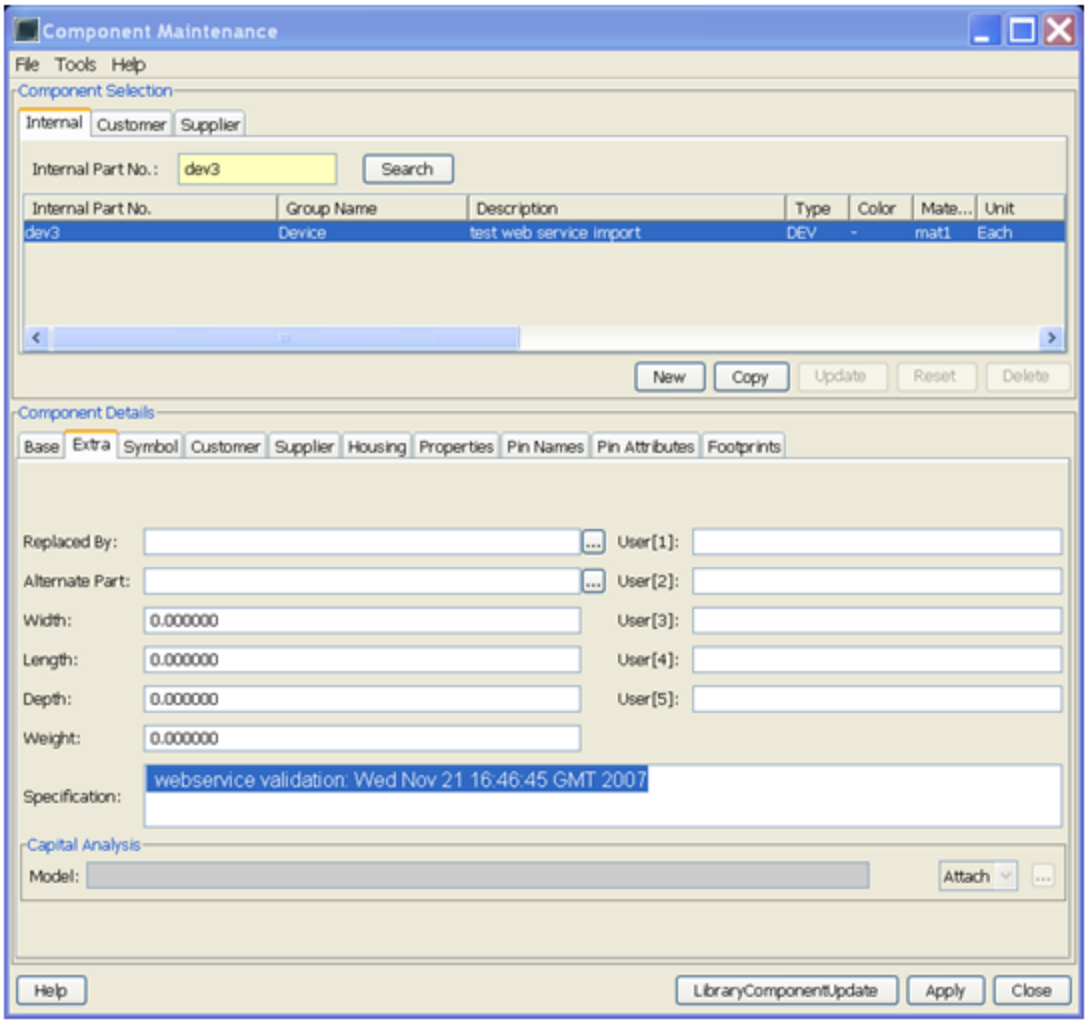
The example web service provided appends a marker string to the “Specification” field of the component.

To invoke this hook service:

- Configure the hook as follows:



- Open the Component Maintenance window in Capital Library, select the Extra tab and click the LibraryComponentUpdate button. The Specification field is updated as follows:



How to implement the library component status change hook

Name	Component Status Change
Request schema	$CAPITAL_HOME/dtd/bridgeslibrrypartstatus.xsd
Response schema	-
Response	-
Transaction type	Asynchronous fire-and-forget

See the provided LibraryComponentStatusChangeHookService example Java class for how to implement this service.

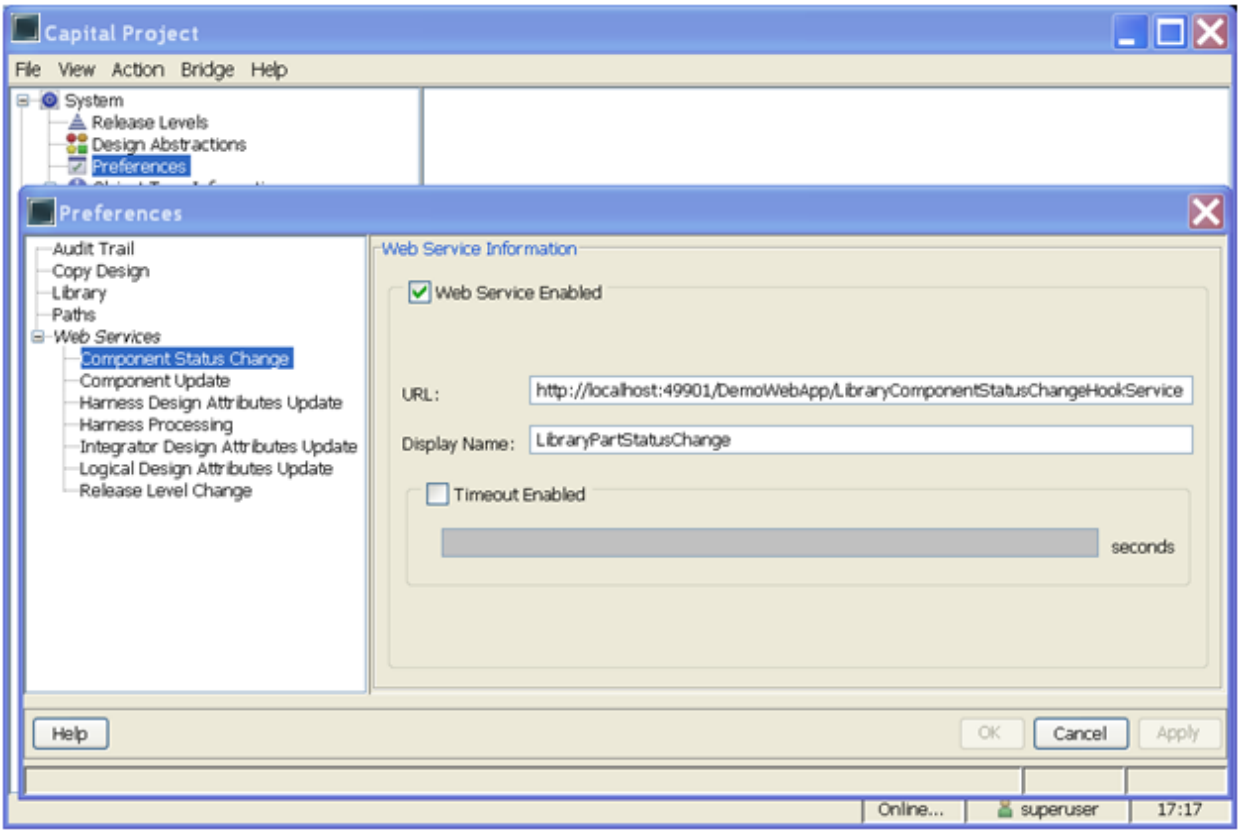
The request payload includes the component identification information, the target status (*currentstatus* attribute) and the current status before transition (*oldstatus* attribute).

```
<?xml version="1.0" encoding="UTF-8"?>
<librarypart xmlns="http://www.mentor.com/harness/Schema/bridgeslibrarypartstatus"
             currentstatus="Current"
             description="test web service import"
             groupname="Device"
             oldstatus="New"
             partnumber="dev3"
             timestamp="Wed Nov 21 17:14:25 GMT 2007"
             typecode="DEV"
             user="system"/>
```

The example web service provided simply outputs the information above in the CIS log file.

To invoke this hook service:

- Configure the hook as follows:



- Open the Component Maintenance window in Capital Library, change the Status field, click the Update and Apply buttons

How to implement the engineering change order release status hook

Name	Engineering Change Order Release Level Change

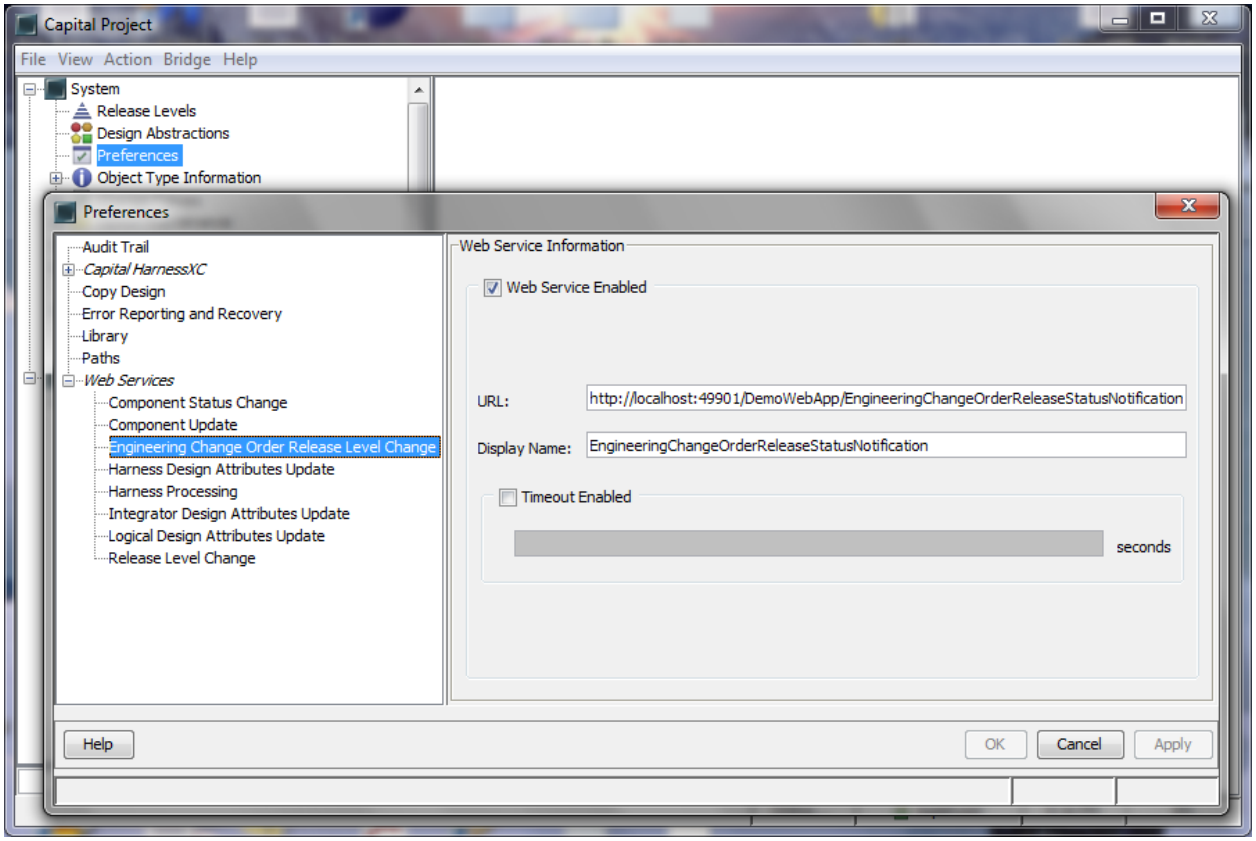
Request schema	$CAPITAL_HOME/dtd/ bridgesengineeringchangeorderstatus.xsd
Response schema	-
Response	-
Transaction type	Asynchronous fire-and-forget

See the provided EngineeringChangeOrderReleaseStatusUpdateHookService example Java class for how to implement this service.

The request payload includes the engineering change order identification, the target release level name and the current (pre-transition) release level name. No response is expected.

To invoke this hook service:

- Configure the hook as follows:



Once activated, the hook will be triggered whenever the release status of the engineering change order is changed.

How to implement the Build List release status hook

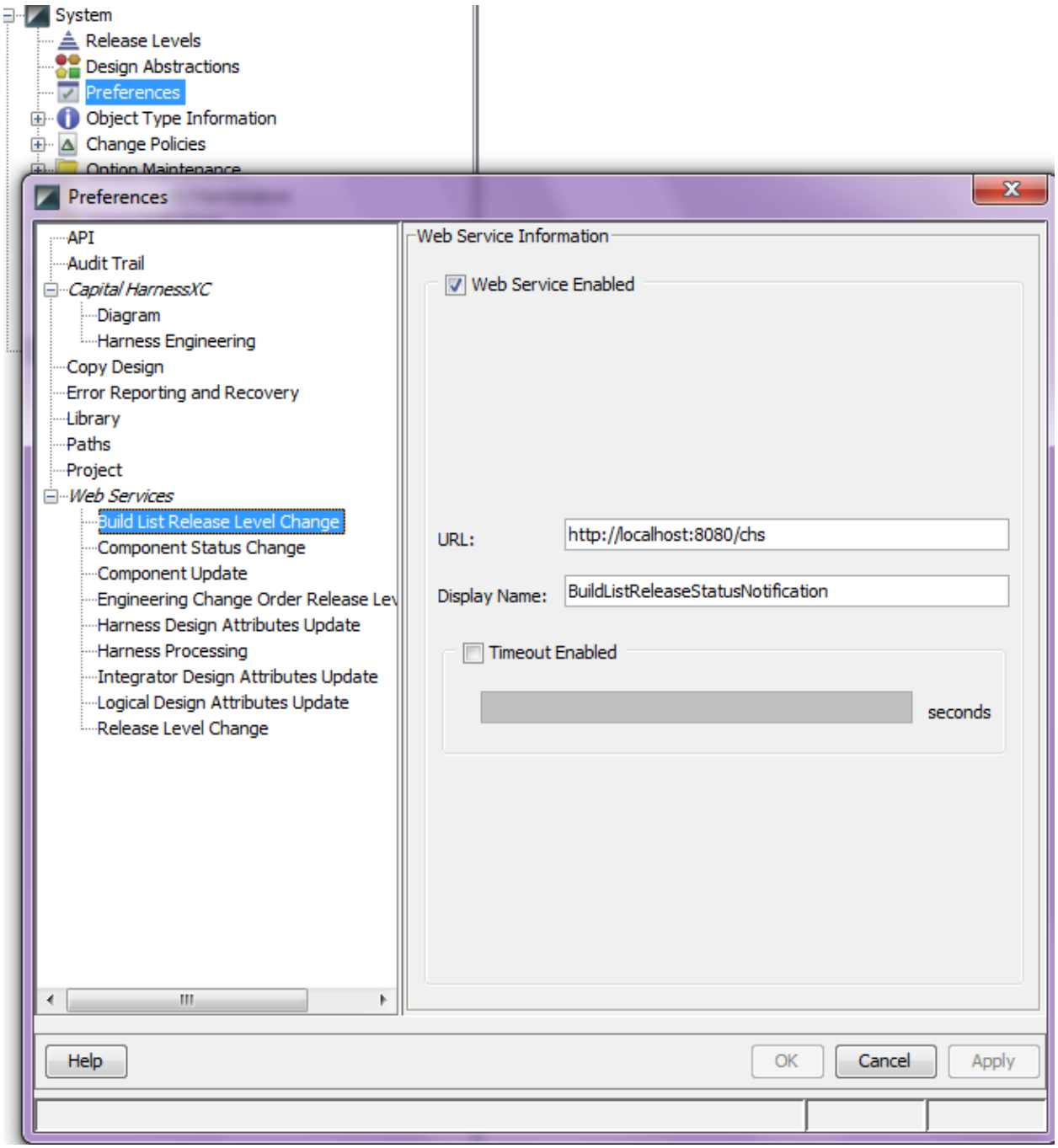
Name	Build List Release Level Change
Request schema	$CAPITAL_HOME/dtd/ bridgesbuildlistreleasestatus.xsd
Response schema	-
Response	-
Transaction type	Asynchronous fire-and-forget

See the provided BuildListReleaseStatusChangeHookService example Java class for how to implement this service.

The request payload includes the old release level of build list and the new release level, build list name , build list type (logic , harness , topology), build list id, build list description (optional) , project name , project id, project description (optional).

To invoke this hook service:

- Configure the hook as follows:



.

Once activated, the hook will be triggered wherever the release status of the build list is changed, including in edit build list or in the CIS build list release status web service.